

CS162
Operating Systems and
Systems Programming
Lecture 17

Caching Wrap-up, Introduction to I/O

Professor Ion Stoica & Matei Zaharia

<https://cs162.org/>

Recall: Page Replacement Policies

- Why do we care about Replacement Policy?
 - Replacement is an issue with any cache
 - Particularly important with pages
 - » The cost of being wrong is high: must go to disk
 - » Must keep important pages in memory, not toss them out
- FIFO (First In, First Out)
 - Throw out oldest page. Be fair – let every page live in memory for same amount of time.
 - Bad – throws out heavily used pages instead of infrequently used
- RANDOM:
 - Pick random page for every replacement
 - Typical solution for TLB's. Simple hardware
 - Pretty unpredictable – makes it hard to make real-time guarantees
- MIN (Minimum):
 - Replace page that won't be used for the longest time
 - Great (provably optimal), but can't really know future...
 - But past is a good predictor of the future ...

Example: FIFO (strawman)

- Suppose we have 3 page frames, 4 virtual pages, and following reference stream:
 - A B C A B D A D B C B
- Consider FIFO Page replacement:

Ref:	A	B	C	A	B	D	A	D	B	C	B
Page:											
1	A					D				C	
2		B					A				
3			C						B		

- FIFO: 7 faults
- When referencing D, replacing A is bad choice, since need A again right away

Example: MIN / LRU

- Suppose we have the same reference stream:
 - A B C A B D A D B C B
- Consider MIN Page replacement:

Ref:	A	B	C	A	B	D	A	D	B	C	B
Page:											
1	A									C	
2		B									
3			C			D					

- MIN: 5 faults
 - Where will D be brought in? Look for page not referenced farthest in future
- What will LRU do?
 - Same decisions as MIN here, but won't always be true!

Is LRU guaranteed to perform well?

- Consider the following: A B C D A B C D A B C D
- LRU Performs as follows (same as FIFO here):

Ref: Page:	A	B	C	D	A	B	C	D	A	B	C	D
1	A			D			C			B		
2		B			A			D			C	
3			C			B			A			D

- Every reference is a page fault!
- Fairly contrived example of working set of $N+1$ on N frames

When will LRU perform badly?

- Consider the following: A B C D A B C D A B C D
- LRU Performs as follows (same as FIFO here):

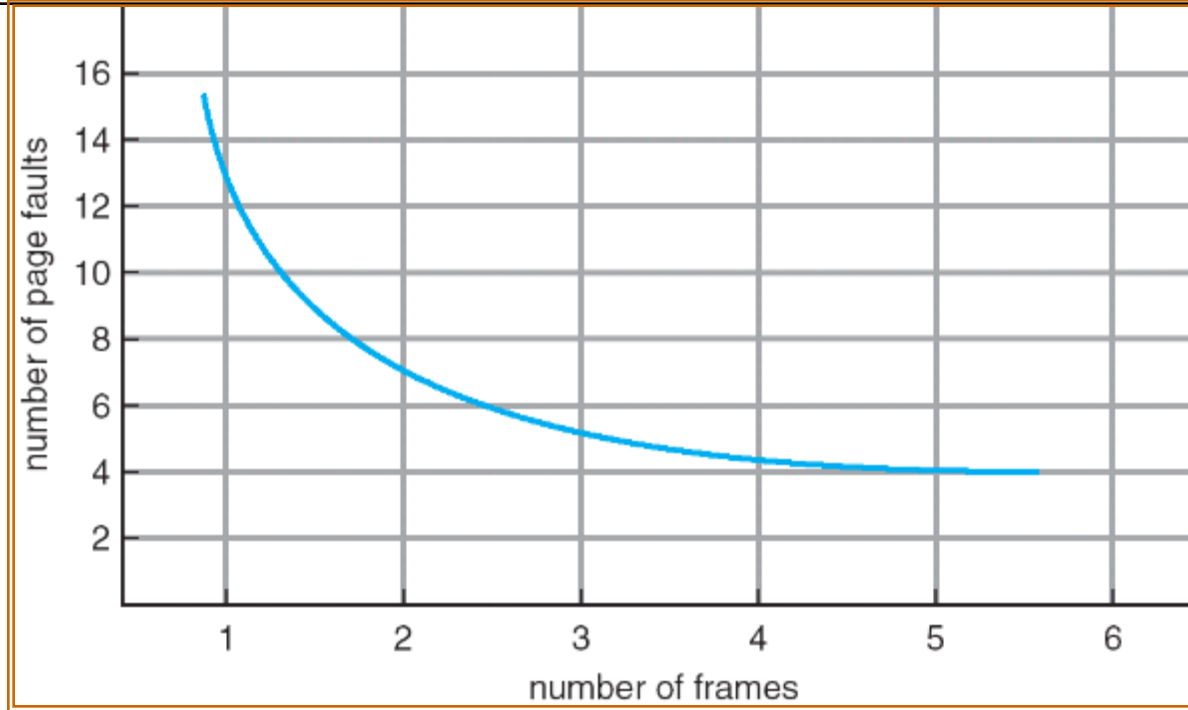
Ref:	A	B	C	D	A	B	C	D	A	B	C	D
Page:												
1	A			D			C			B		
2		B			A			D			C	
3			C			B			A			D

– Every reference is a page fault!

- MIN Does much better:

Ref:	A	B	C	D	A	B	C	D	A	B	C	D
Page:												
1	A									B		
2		B					C					
3			C	D								

Graph of Page Faults Versus The Number of Frames



- One desirable property: When you add memory the miss rate drops (stack property)
 - Does this always happen?
 - Seems like it should, right?
- No: Bélády's anomaly
 - Certain replacement algorithms (FIFO) don't have this obvious property!

Adding Memory Doesn't Always Help Fault Rate

- Does adding memory reduce number of page faults?
 - Yes for LRU and MIN
 - Not necessarily for FIFO! (Called Bélády's anomaly)

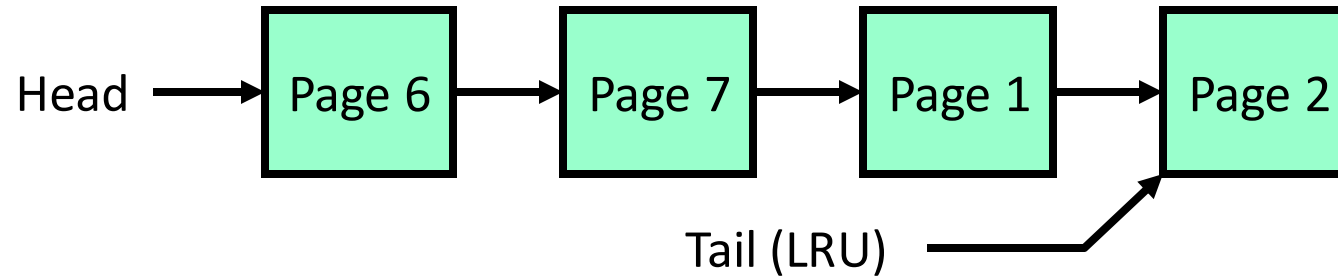
Ref: Page:	A	B	C	D	A	B	E	A	B	C	D	E
1	A			D			E					
2		B			A					C		
3			C			B					D	

Ref: Page:	A	B	C	D	A	B	E	A	B	C	D	E
1	A						E				D	
2		B						A				E
3			C						B			
4				D						C		

- After adding memory:
 - With FIFO, contents can be completely different
 - In contrast, with LRU or MIN, contents of memory with X pages are a subset of contents with $X+1$ Page

LRU implementation

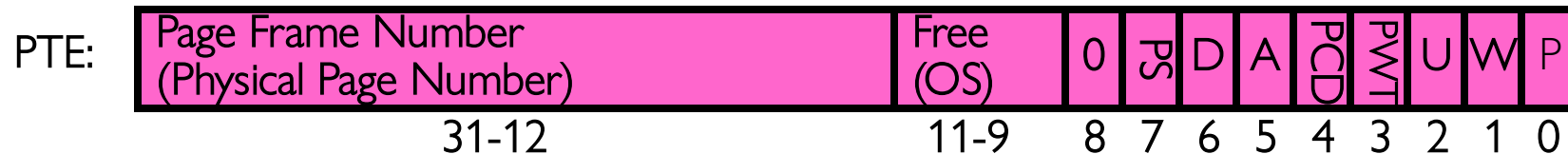
- How to implement LRU? Use a list:



- On each use, remove page from list and place at head
 - LRU page is at tail
- Problems with this scheme for paging?
 - Need to know immediately when page used so that can change position in list...
 - Many instructions for each hardware access
- In practice, people **approximate** LRU (more later)

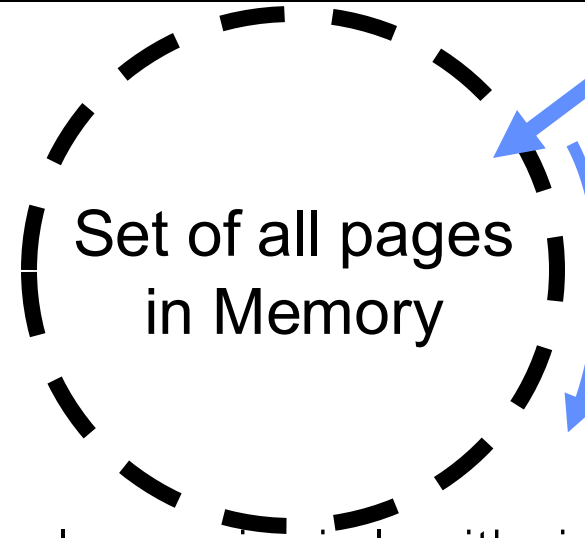
Approximating LRU: Recall PTE bits

- Which bits of a PTE entry can help us approximate LRU?
Remember Intel PTE:



- The “Present” bit (called “Valid” elsewhere):
 - » $P=0$: Page is invalid and a reference will cause page fault
 - » $P=1$: Page frame number is valid and MMU is allowed to proceed with translation
- The “Writable” bit (could have opposite sense and be called “Read-only”):
 - » $W=0$: Page is read-only and cannot be written.
 - » $W=1$: Page can be written
- The “Accessed” bit (called “Use” elsewhere):
 - » $A=0$: Page has not been accessed (or used) since last time software set $A \rightarrow 0$
 - » $A=1$: Page has been accessed (or used) since last time software set $A \rightarrow 0$
- The “Dirty” bit (called “Modified” elsewhere):
 - » $D=0$: Page has not been modified (written) since PTE was loaded
 - » $D=1$: Page has changed since PTE was loaded

Approximating LRU: Clock Algorithm



Single Clock Hand:

Advances only on page fault!

Check for pages not used recently

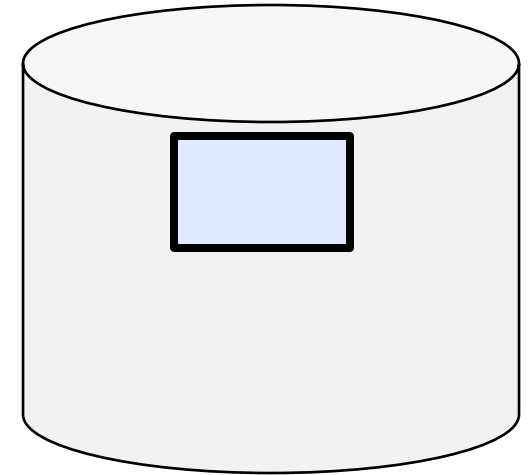
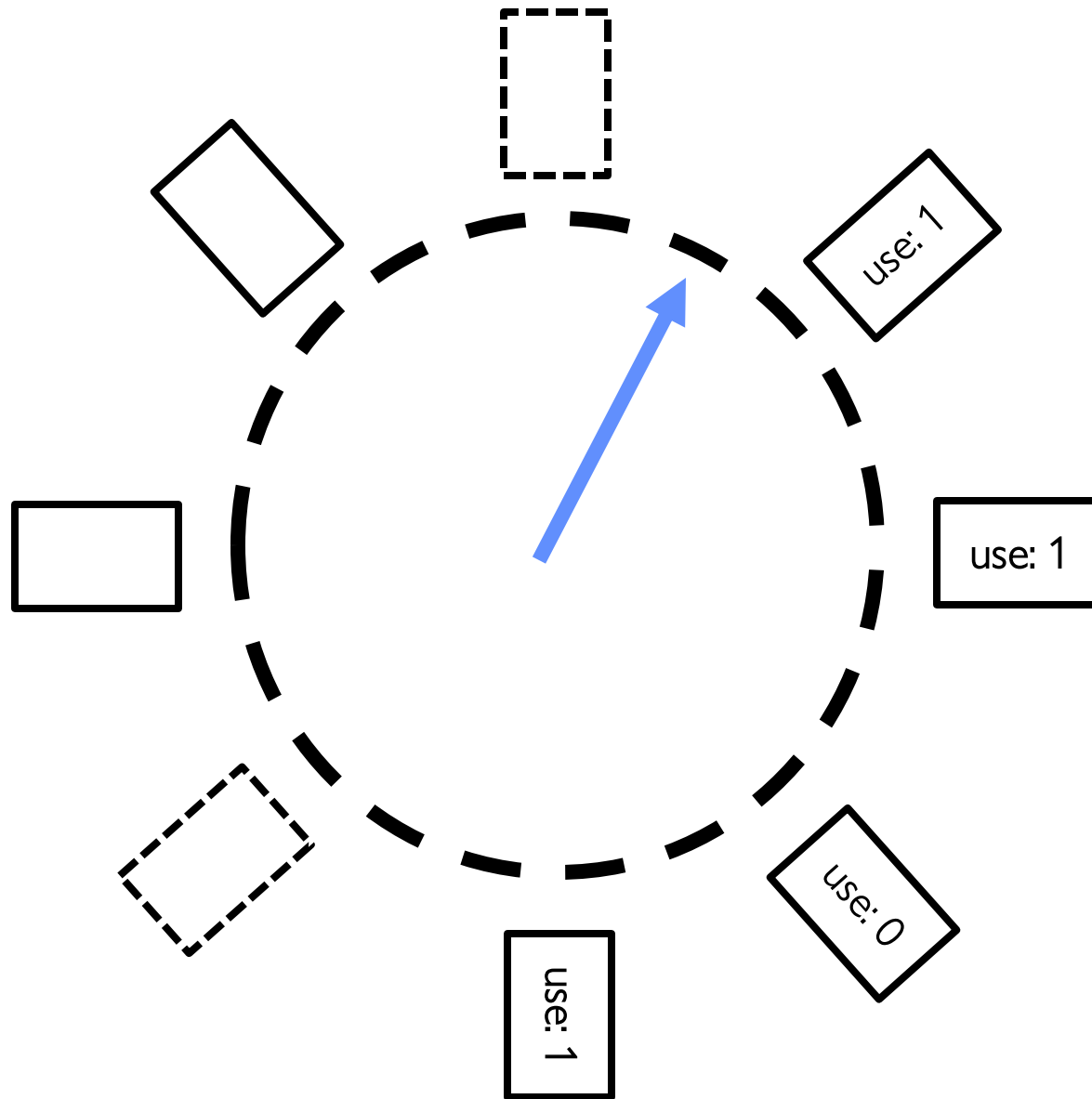
Mark pages as not used recently

- **Clock Algorithm:** Arrange physical pages in circle with single clock hand
 - Approximate LRU (approximation to approximation to MIN)
 - Replace **an** old page, not **the oldest** page
- Details:
 - Hardware “**use**” bit per physical page (called “**accessed**” in Intel architecture):
 - » Hardware sets **use** bit on each reference
 - » If **use** bit isn’t set, means not referenced in a long time
 - » Some hardware sets **use** bit in the TLB; must be copied back to PTE when TLB entry gets replaced
 - On page fault:
 - » Advance clock hand (not real time)
 - » Check **use** bit: 1→ used recently; clear and leave alone
0→ selected candidate for replacement

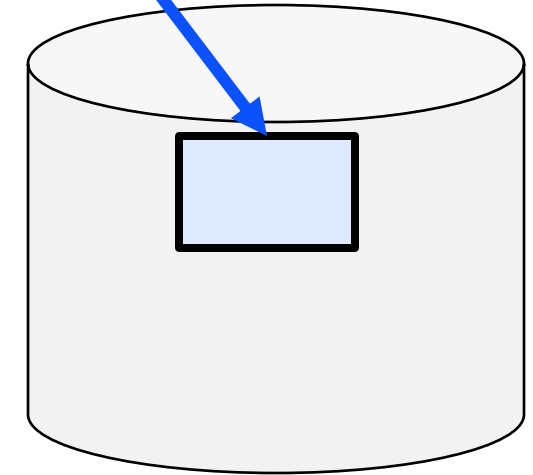
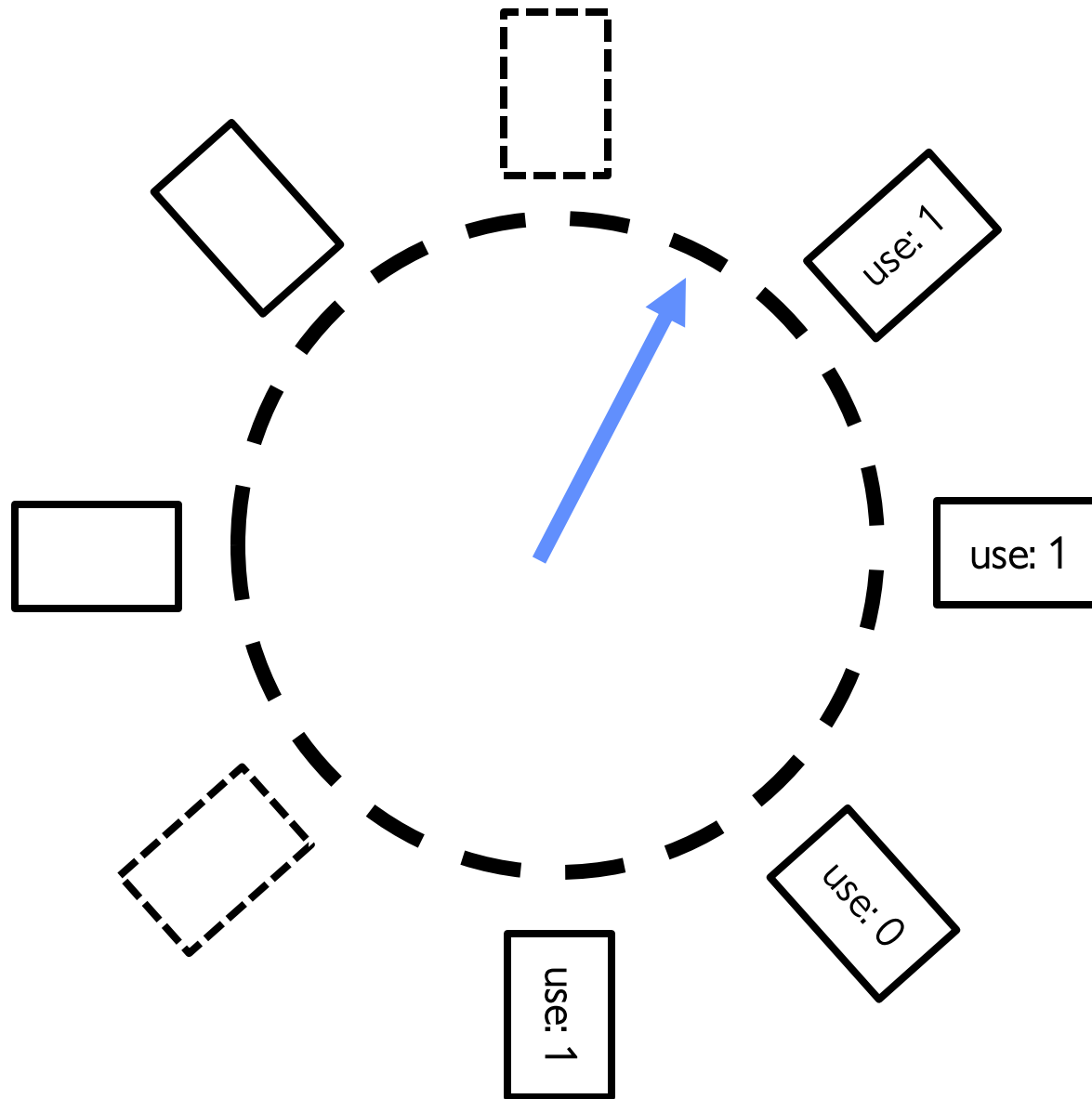
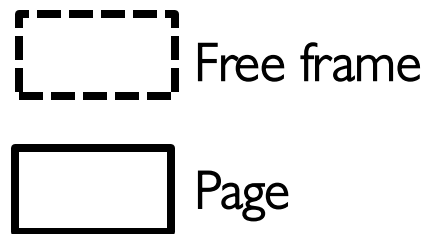
Clock Algorithm Example

 Free frame

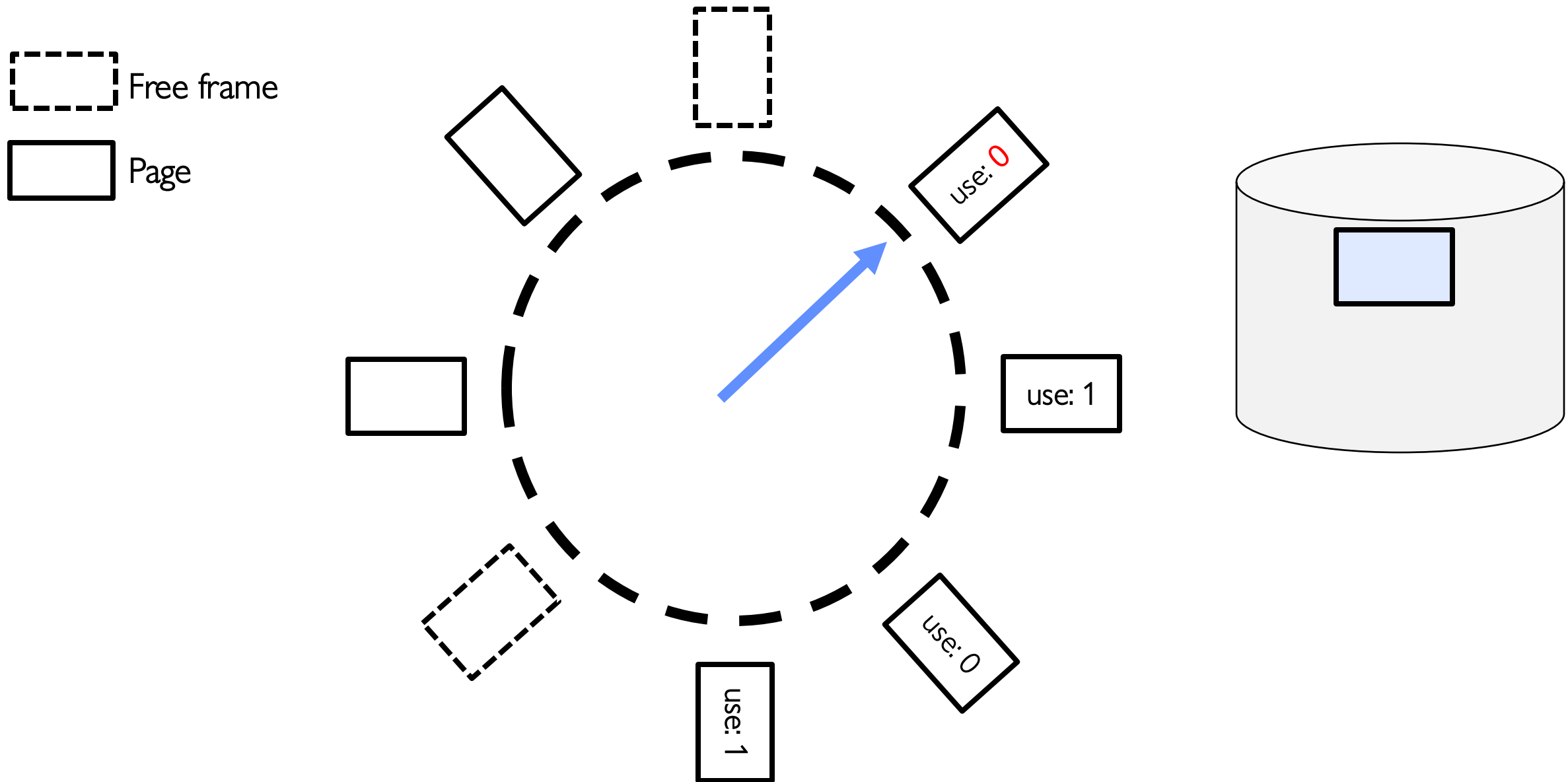
 Page



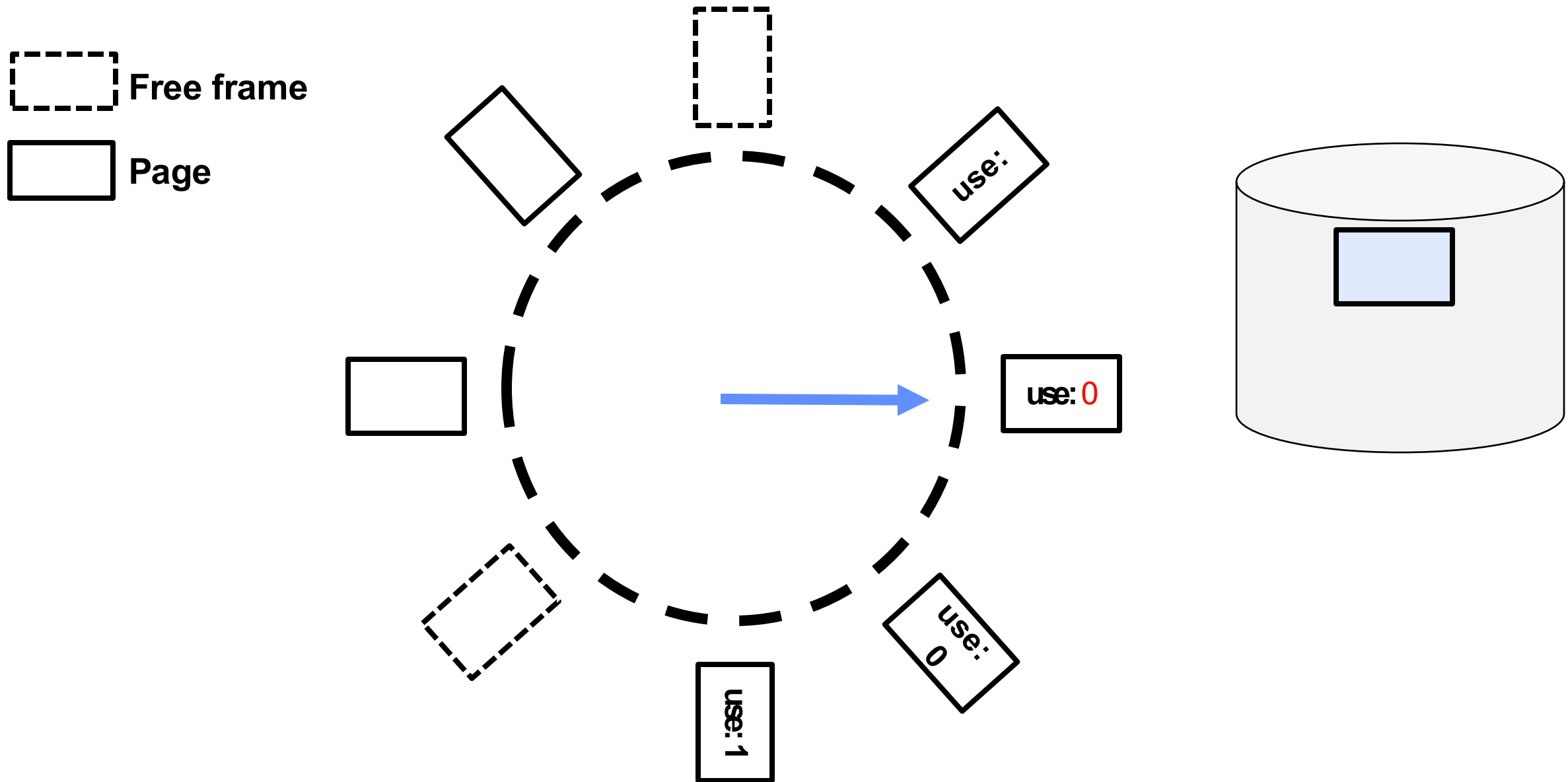
Clock Algorithm Example: Page Fault



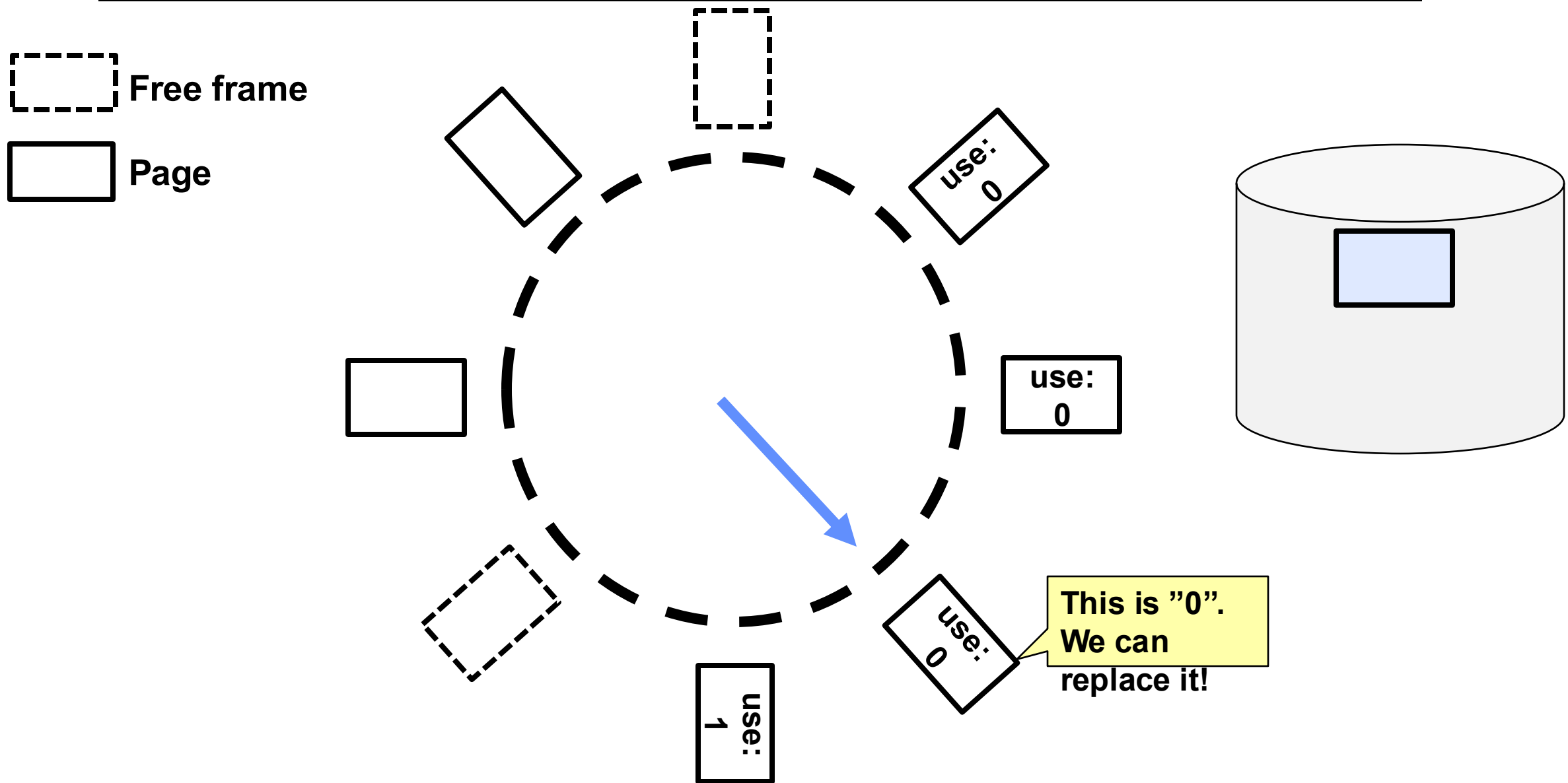
Clock Algorithm Example: Page Fault



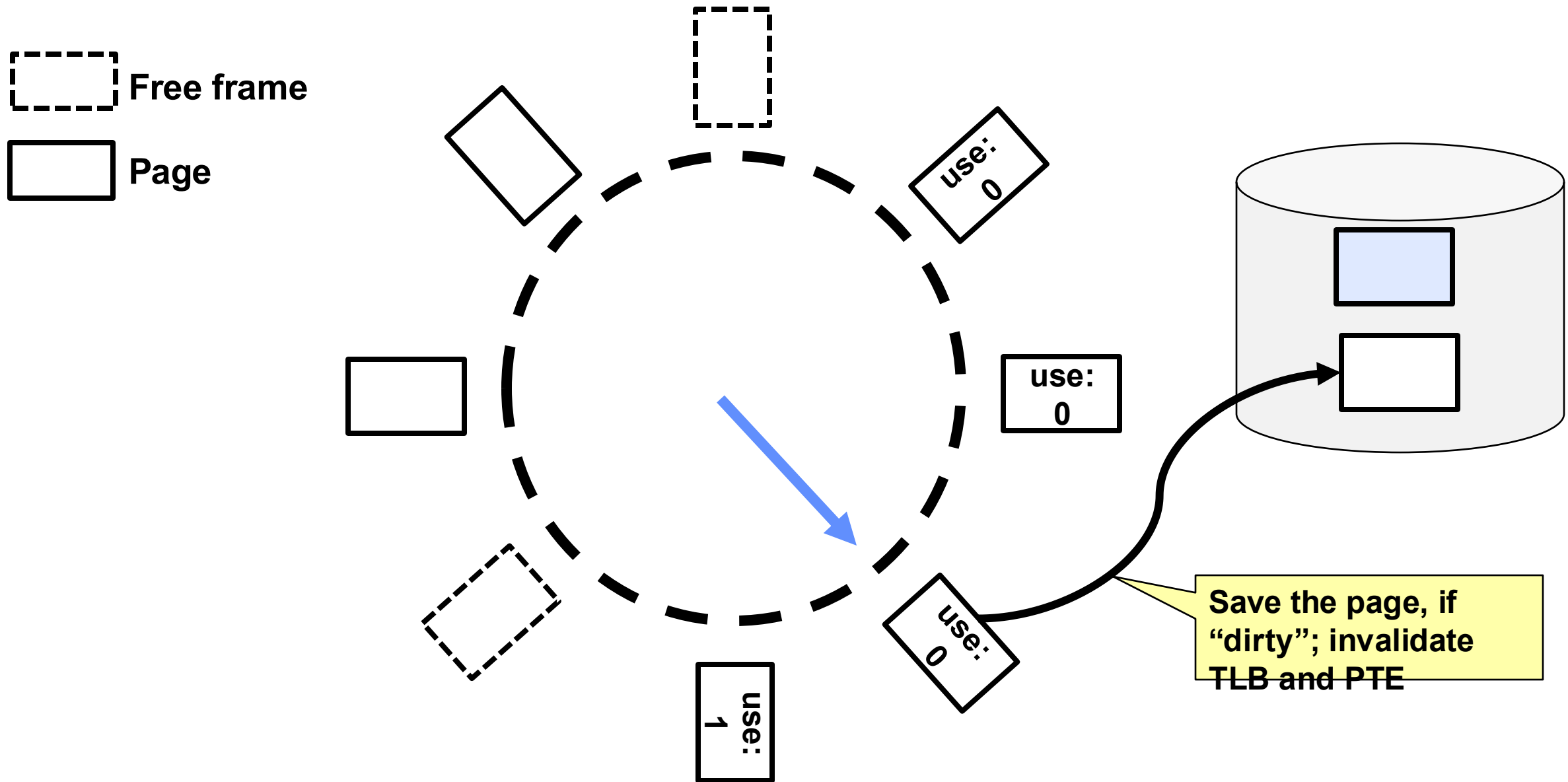
Clock Algorithm Example: Page Fault



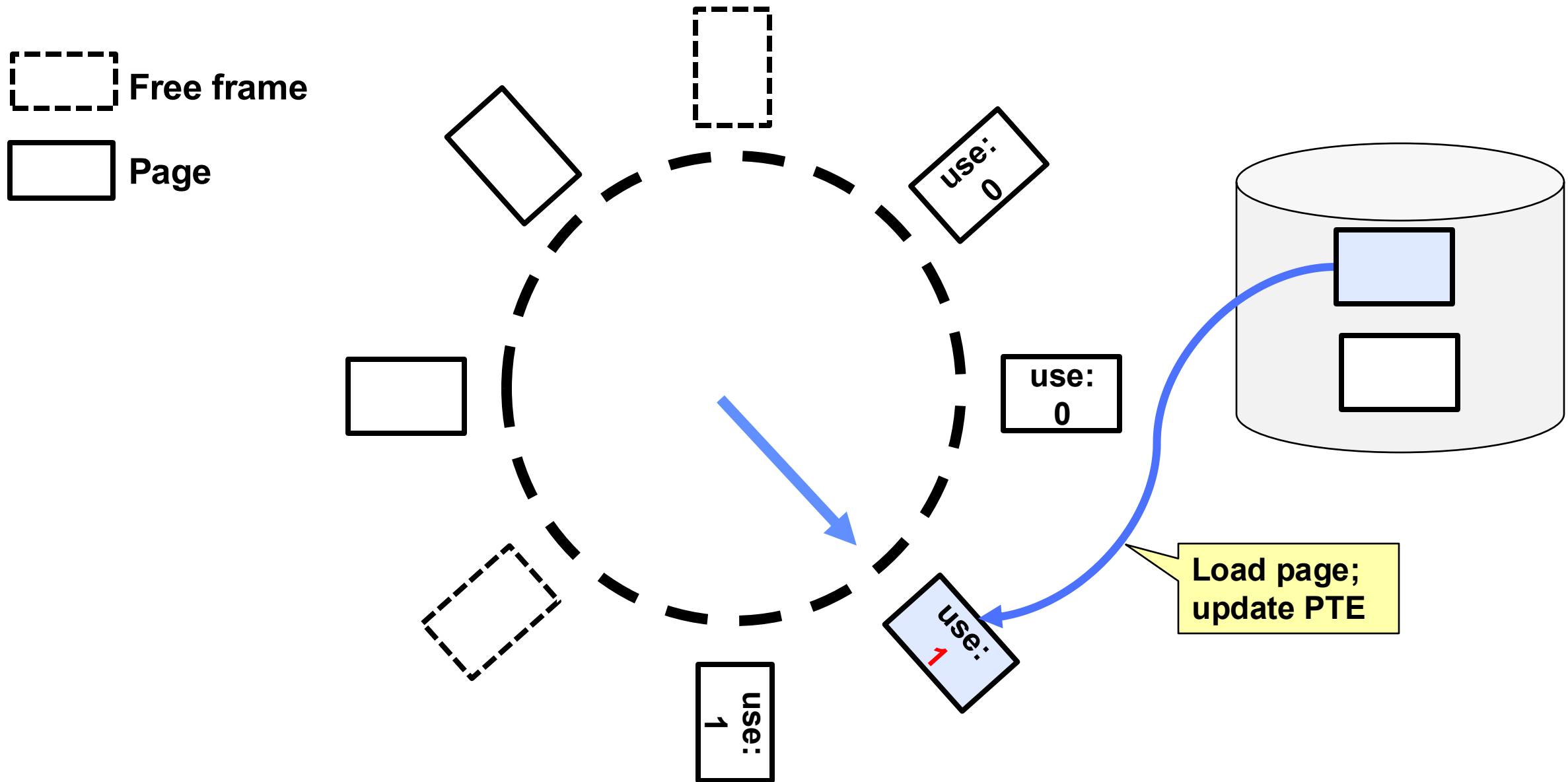
Clock Algorithm Example: Page Fault



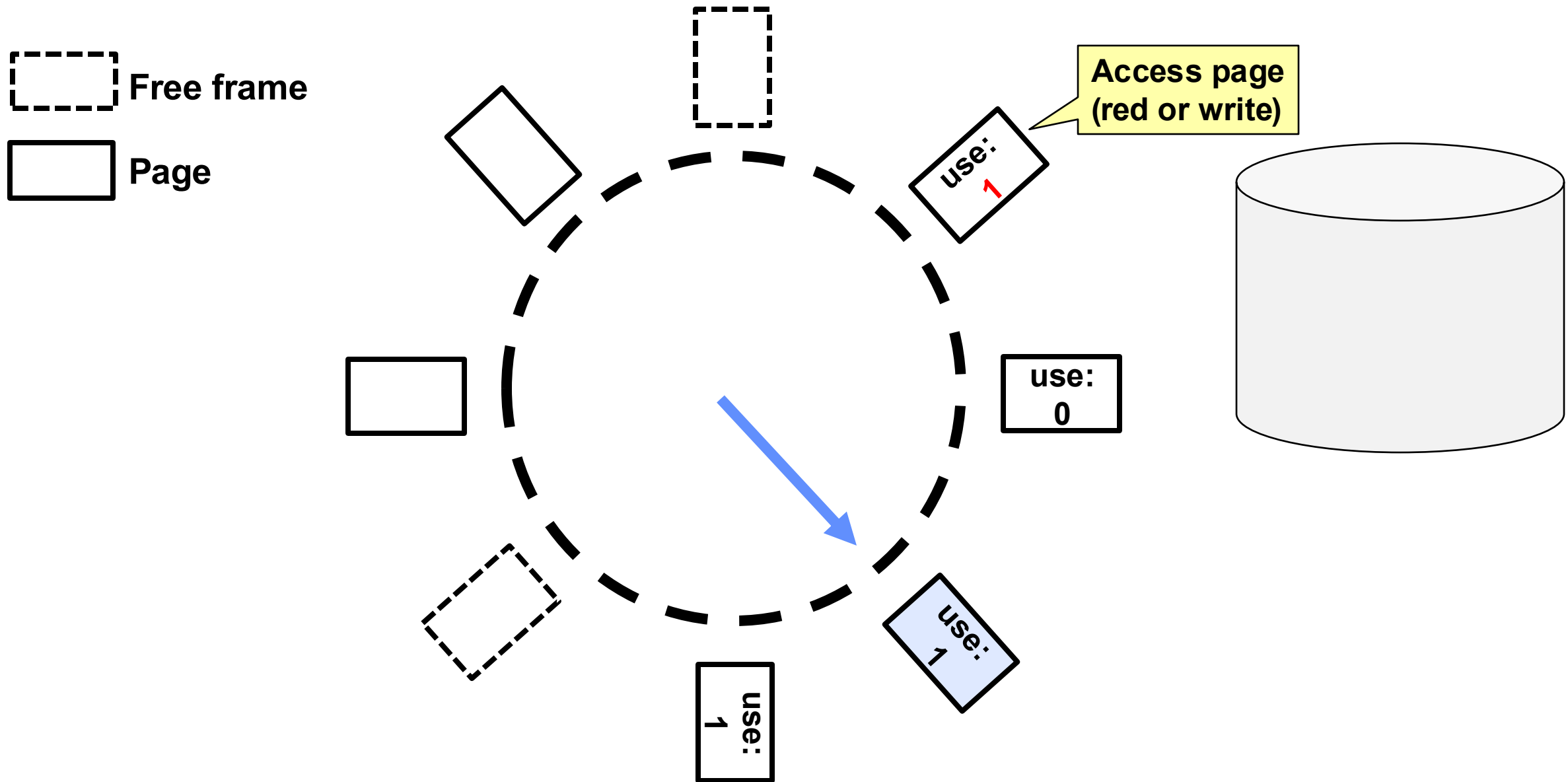
Clock Algorithm Example: Page Fault



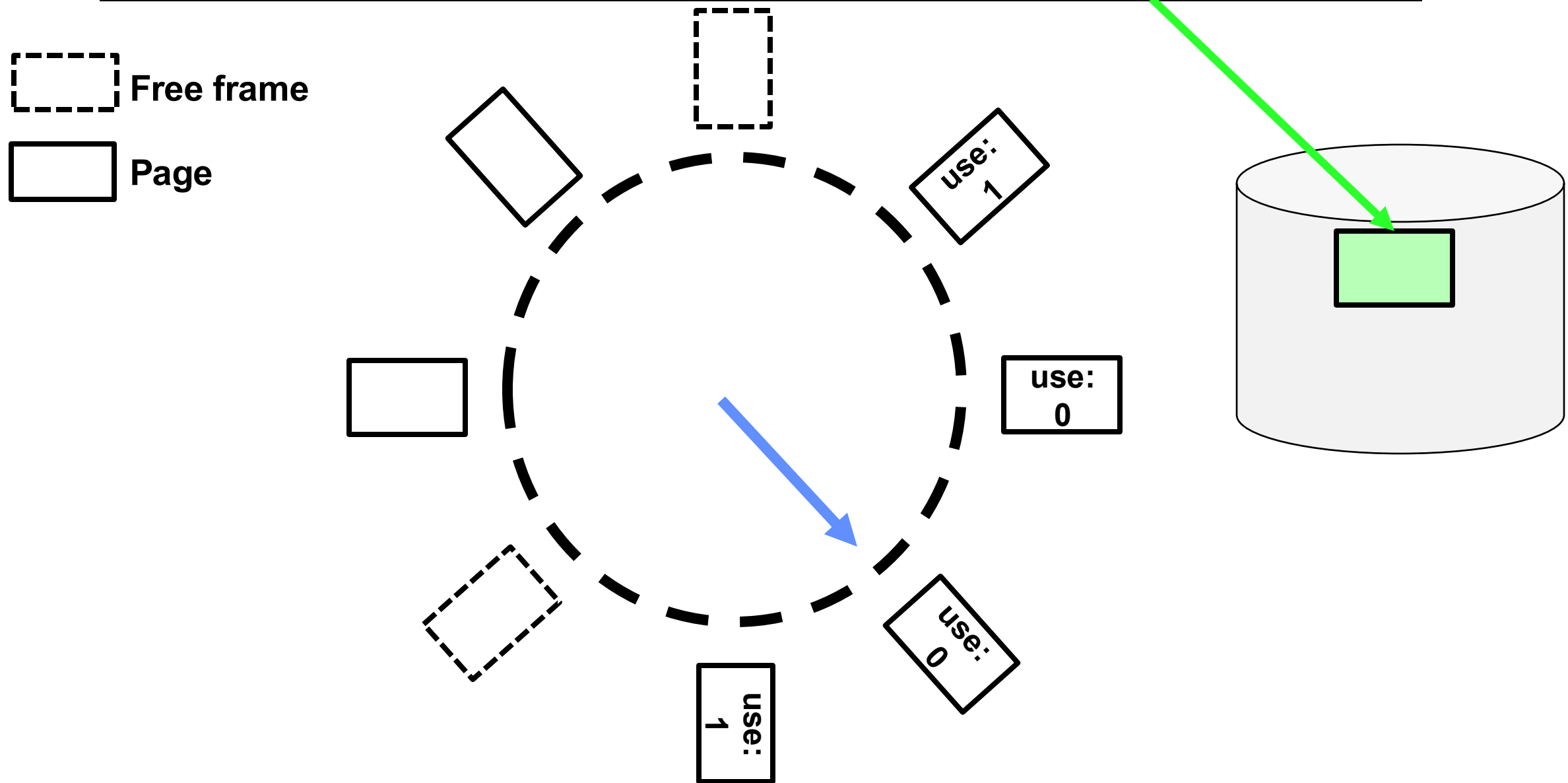
Clock Algorithm Example: Page Fault



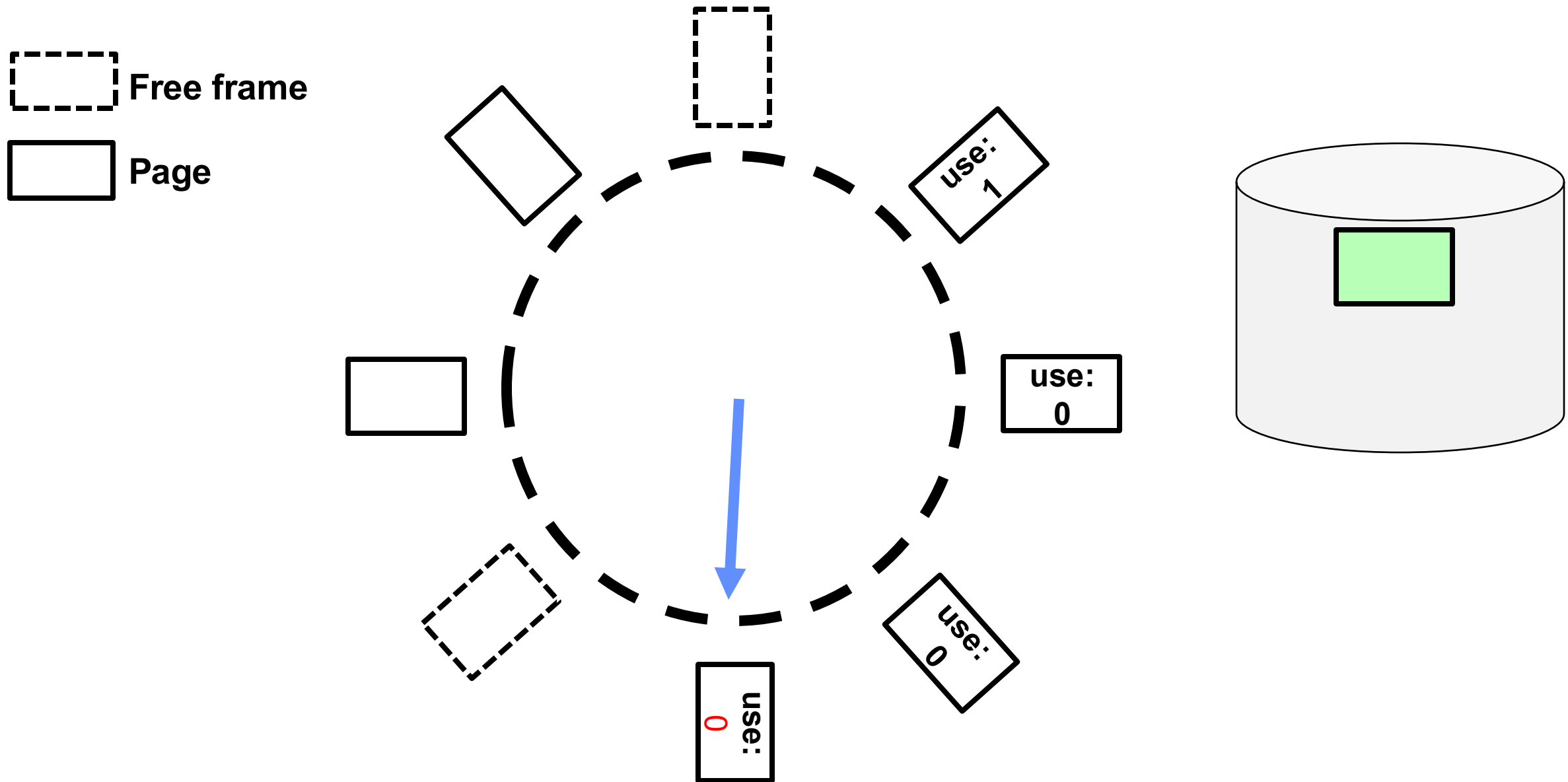
Clock Algorithm Example



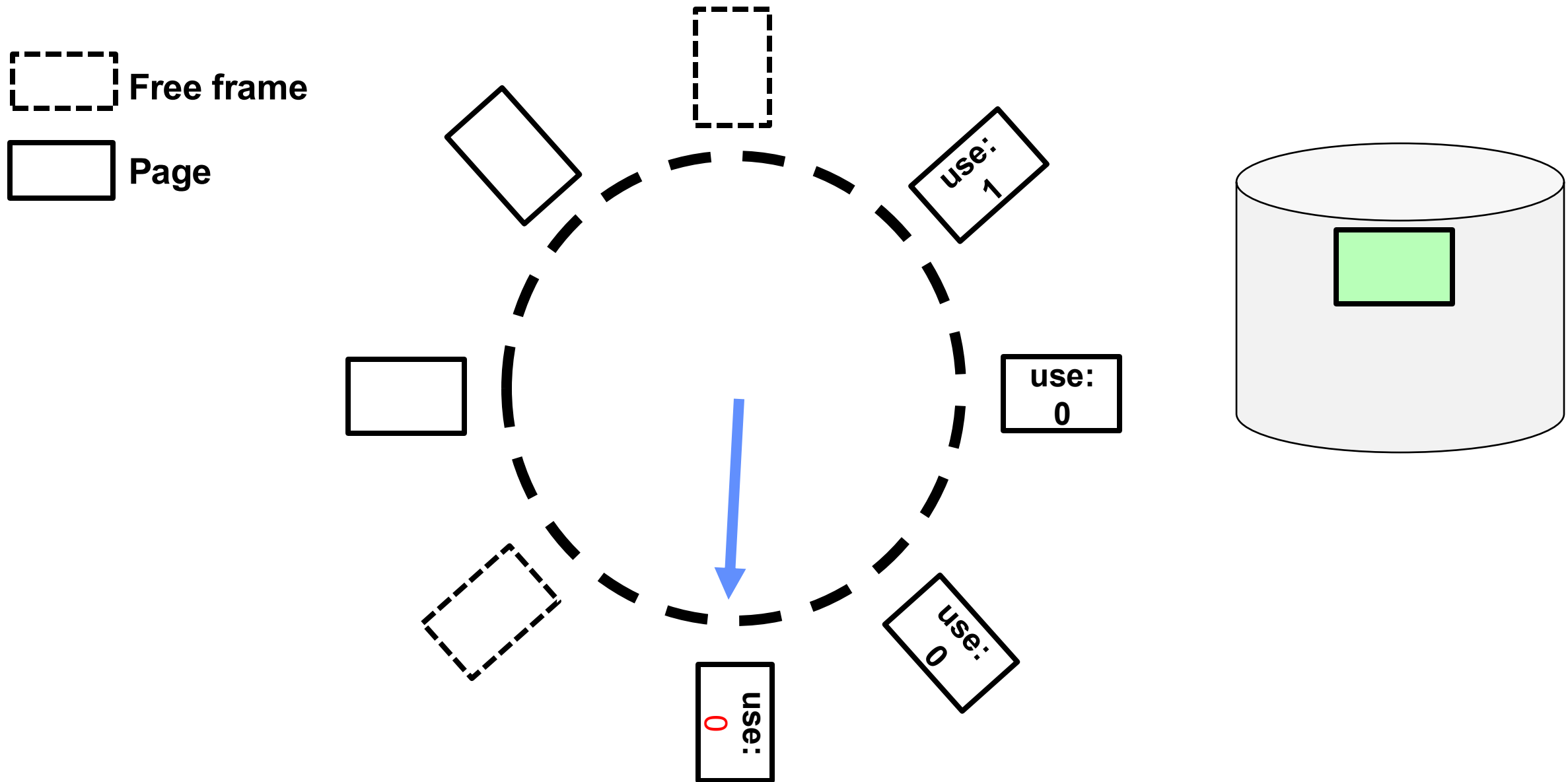
Clock Algorithm Example: Another Page Fault



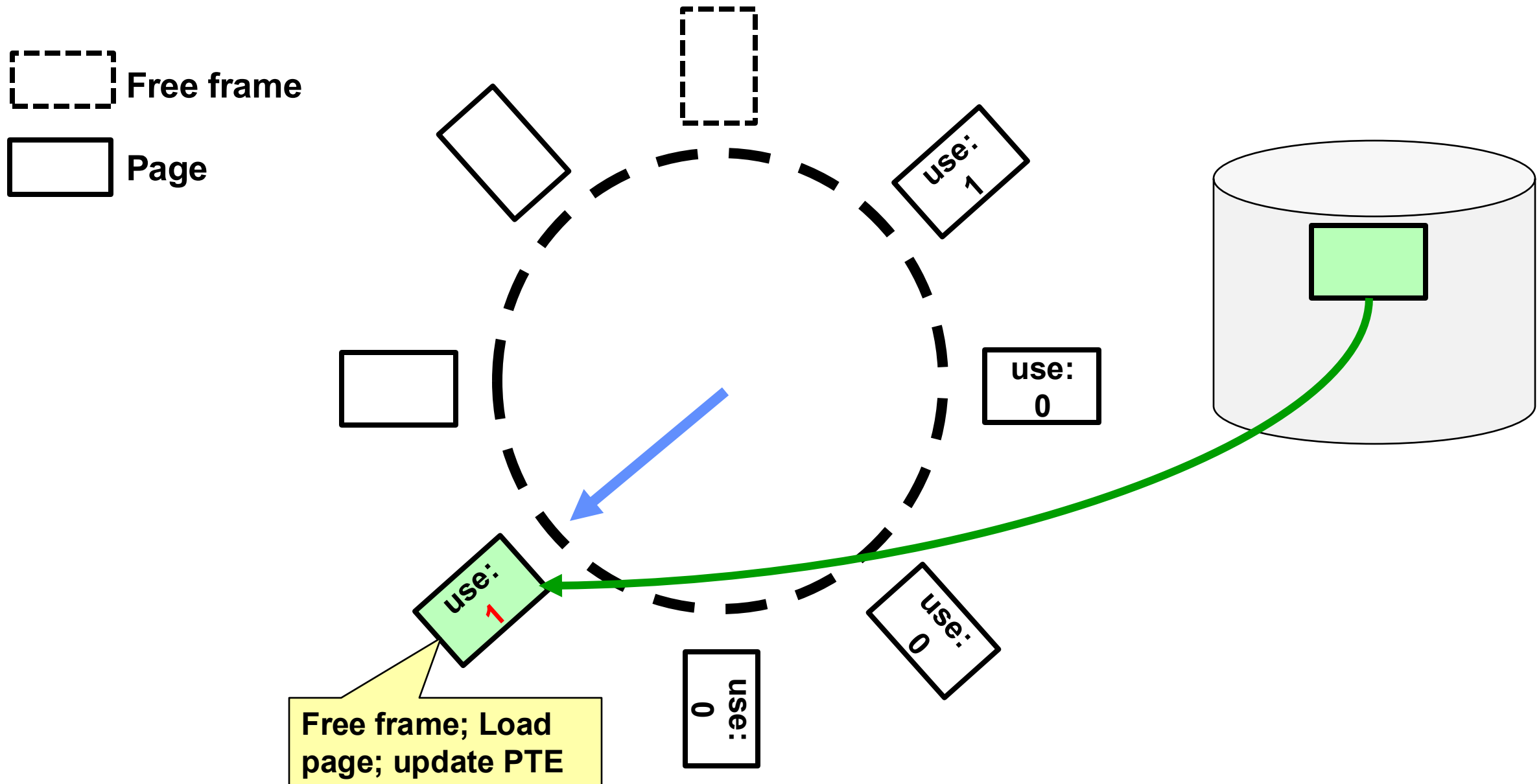
Clock Algorithm Example: Another Page Fault



Clock Algorithm Example: Another Page Fault

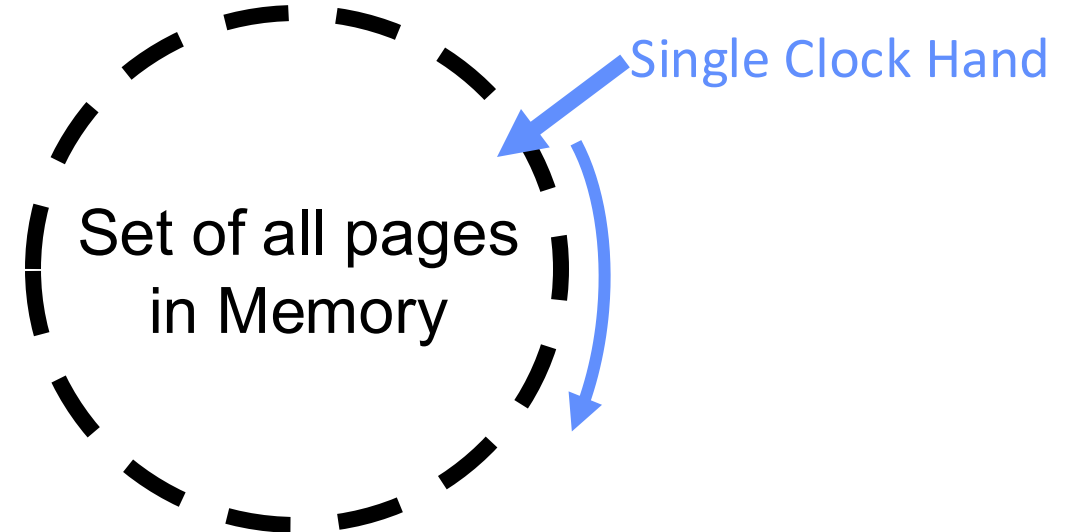


Clock Algorithm Example: Another Page Fault



Clock Algorithm: More details

- Will always find a page or loop forever?
 - Even if all use bits set, will eventually loop all the way around $\Rightarrow$ FIFO
- What if hand moving slowly?
 - Good sign or bad sign?
 - » Not many page faults
 - » or find page quickly
- What if hand is moving quickly?
 - Lots of page faults and/or lots of reference bits set
- One way to view clock algorithm:
 - Crude partitioning of pages into two groups: young and old



Nth Chance version of Clock Algorithm

- **Nth chance algorithm:** Give page N chances
 - OS keeps counter per page: # sweeps
 - On page fault, OS checks use bit:
 - » 1 → clear use and also clear counter (used in last sweep)
 - » 0 → increment counter; if count=N, replace page
 - Means that clock hand might have to sweep by N times without page being used before page is replaced
- How do we pick N?
 - Why pick large N? Better approximation to LRU
 - » If $N \sim 1K$, really good approximation
 - Why pick small N? More efficient
 - » Otherwise, might have to look a long way to find free page
- What about “**modified**” (or “**dirty**”) pages?
 - Takes extra overhead to replace a dirty page, so give dirty pages an extra chance before replacing?
 - Common approach:
 - » Clean pages, use $N=1$
 - » Dirty pages, use $N=2$ (and write back to disk when $N=1$)

Clock Algorithms Variations

- Do we really need hardware-supported “modified” (“dirty”) bit?
 - No. We can emulate it using read-only bit
 - » Need software “database” of which pages are allowed to be written (needed this anyway)
 - » We will tell MMU that pages have more restricted permissions than the actually do to force page faults (and allow us notice when page is written)
 - Algorithm (Clock-Emulated-M):
 - » Initially, mark all pages as read-only ($W \rightarrow 0$), even writable data pages.
Further, clear all software versions of the “modified” bit $\rightarrow 0$ (page not dirty)
 - » Writes will cause a page fault. Assuming write is allowed, OS sets software “modified” bit $\rightarrow 1$, and marks page as writable ($W \rightarrow 1$).
 - » Whenever page written back to disk, clear “modified” bit $\rightarrow 0$, mark read-only

Clock Algorithms Variations (continued)

- Do we really need a hardware-supported “**use**” bit?
 - No. Can emulate it similar to above (e.g. for read operation)
 - » Kernel keeps a “**use**” bit and “**modified**” bit for each page
 - Algorithm (Clock-Emulated-Use-and-M):
 - » Mark all pages as **invalid**, even if in memory.
Clear emulated “**use**” bits $\rightarrow 0$ and “**modified**” bits $\rightarrow 0$ for all pages (not used, not dirty)
 - » Read or write to invalid page traps to OS to tell use page has been used
 - » OS sets “**use**” bit $\rightarrow 1$ in software to indicate that page has been “used”.
Further:
 - 1) If read, mark page as read-only, $W \rightarrow 0$ (will catch future writes)
 - 2) If write (and write allowed), set “**modified**” bit $\rightarrow 1$, mark page as writable ($W \rightarrow 1$)
 - » When clock hand passes, reset emulated “**use**” bit $\rightarrow 0$ and mark page as invalid again
 - » Note that “**modified**” bit left alone until page written back to disk
- Remember, however, clock is just an approximation of LRU!
 - Can we do a better approximation, given that we have to take page faults on some reads and writes to collect use information?
 - Need to identify an old page, not oldest page!

Conclusion

- Replacement policies
 - FIFO: Place pages on queue, replace page at end
 - MIN: Replace page that will be used farthest in future
 - LRU: Replace page used farthest in past
- Working Set:
 - Set of pages touched by a process recently
 - Point of Replacement algorithms is to try to keep working set in memory
- Clock Algorithm: Approximation to LRU
 - Arrange all pages in circular list
 - Sweep through them, marking as not “in use”
 - If page not “in use” for one pass, than can replace
- Nth-chance clock algorithm: Another approximate LRU
 - Give pages multiple passes of clock hand before replacing

CS162
Operating Systems and
Systems Programming
Lecture 17

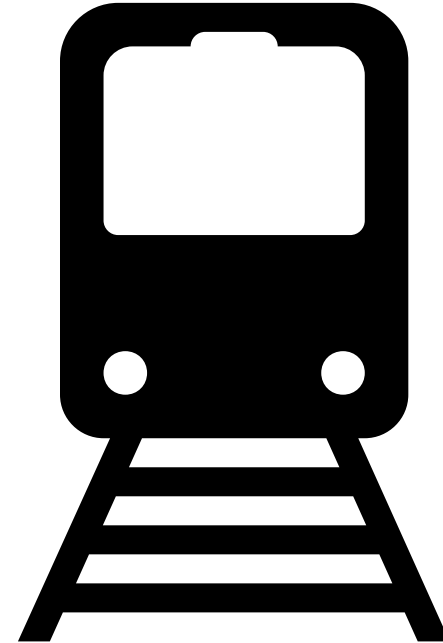
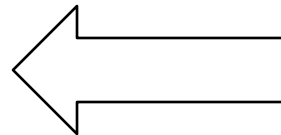
Introduction to I/O

Professor Ion Stoica & Matei Zaharia

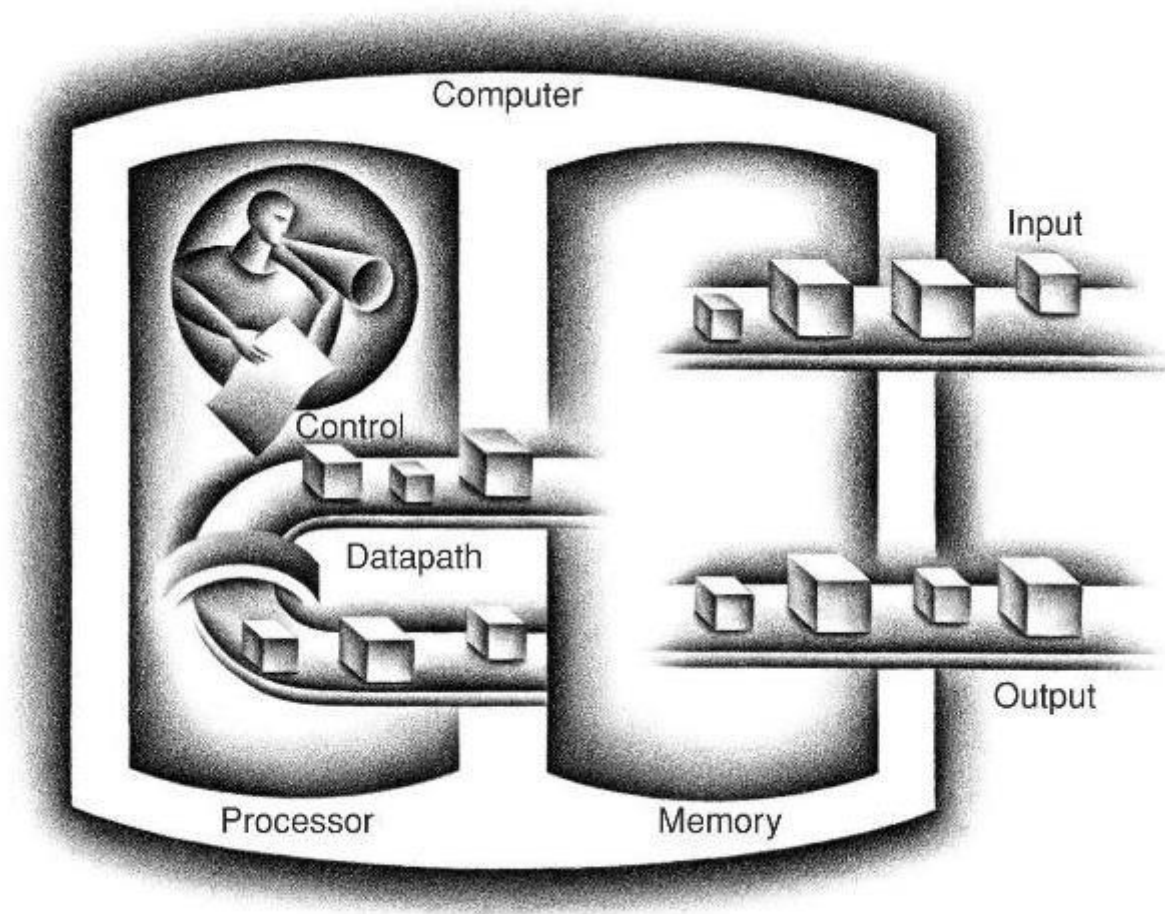
<https://cs162.org/>

Course Map

- Introduction
- OS Concepts
- Concurrency
- Scheduling
- Memory Management
- Devices and file systems
- Reliability, networking and cloud



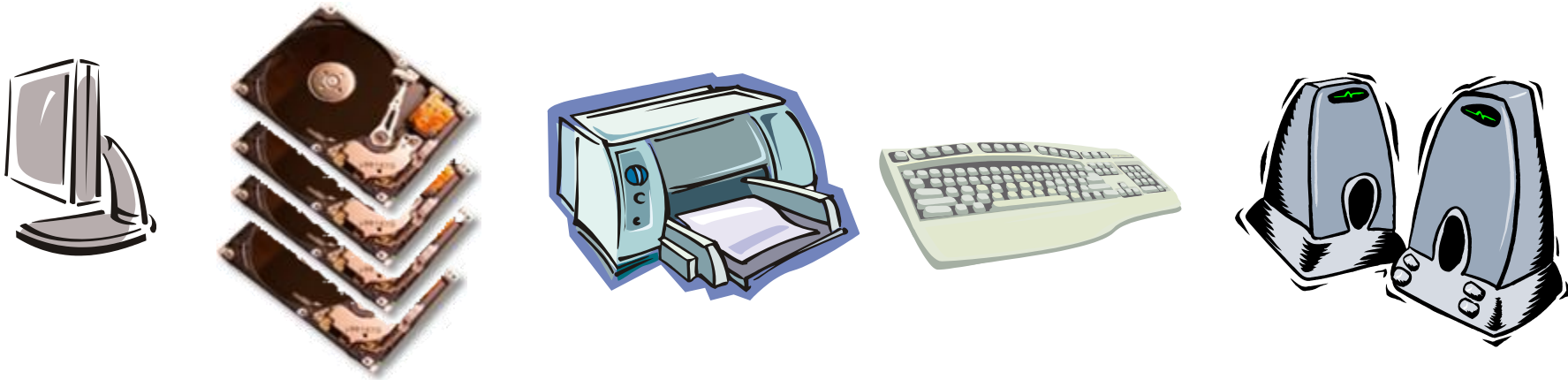
Recall: Five Components of a Computer



From "Computer Organization and Design" by Hennesy & Patterson

CPU: You need to get out more!

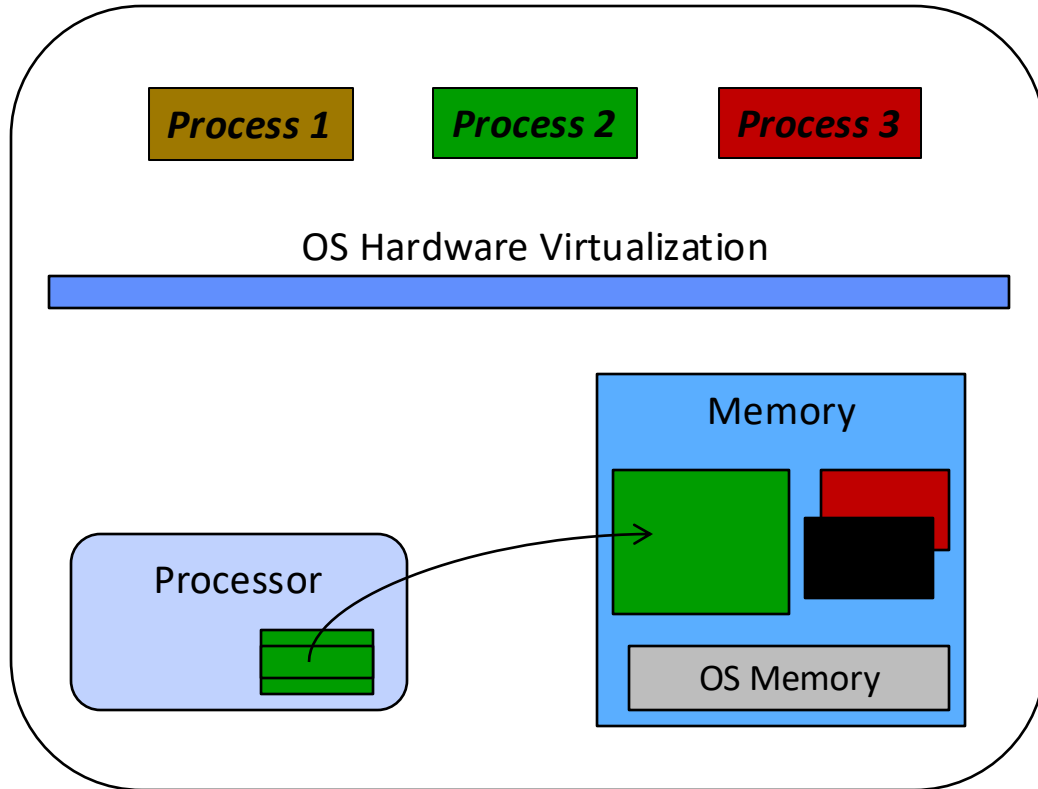
Input/output is the mechanism through which the computer communicates with the outside world



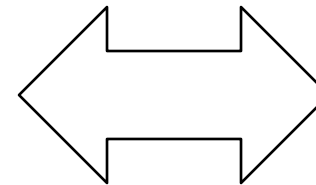
Want Standard Interfaces to Devices

- **Block Devices:** *e.g.* disk drives, tape drives, DVD-ROM
 - Access blocks of data
 - Commands include `open()`, `read()`, `write()`, `seek()`
 - Raw I/O or file-system access
- **Character Devices:** *e.g.* keyboards, mice, serial ports, some USB devices
 - Single characters at a time
 - Commands include `get()`, `put()`
- **Network Devices:** *e.g.* Ethernet, Wireless, Bluetooth
 - Different enough from block/character to have own interface
 - Unix and Windows include **socket** interface
 - » Separates network protocol from network operation
 - » Includes polling and connection management functionality
 - Usage: pipes, FIFOs, streams, queues, mailboxes

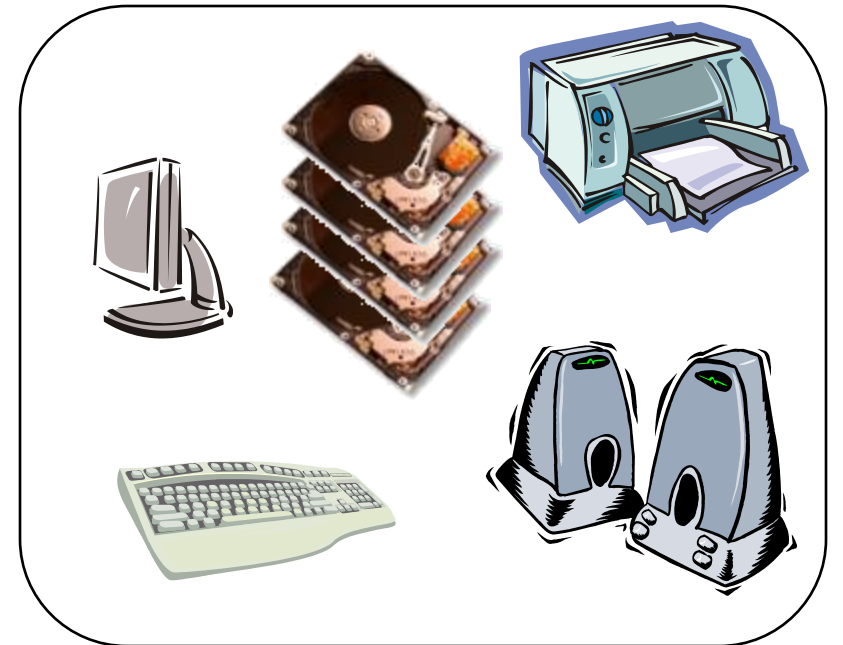
IO Subsystem: Abstraction, abstraction, abstraction



Virtual Machine Abstraction



IO Layer



IO Devices

IO Subsystem: Abstraction, abstraction, abstraction

- This code works for pretty much any device

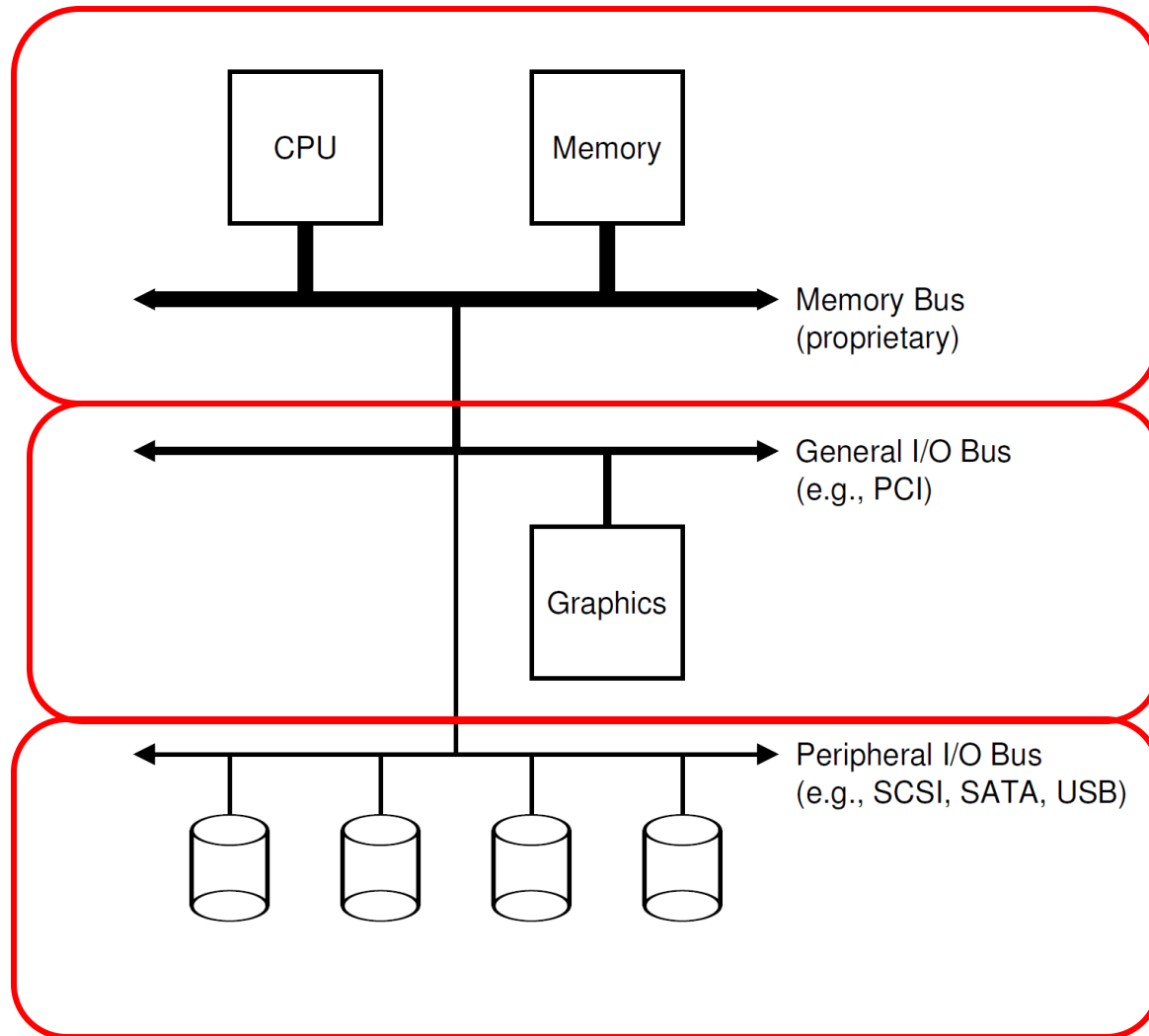
```
FILE fd = fopen("/dev/something", "rw");
for (int i = 0; i < 10; i++) {
    fprintf(fd, "Count %d\n", i);
}
close(fd);
```

- Why? Because code that controls devices (“device driver”) implements standard interface
- We will try to get a flavor for what is involved in actually controlling devices in rest of this lecture
 - Can only scratch surface!

Requirements of I/O layer

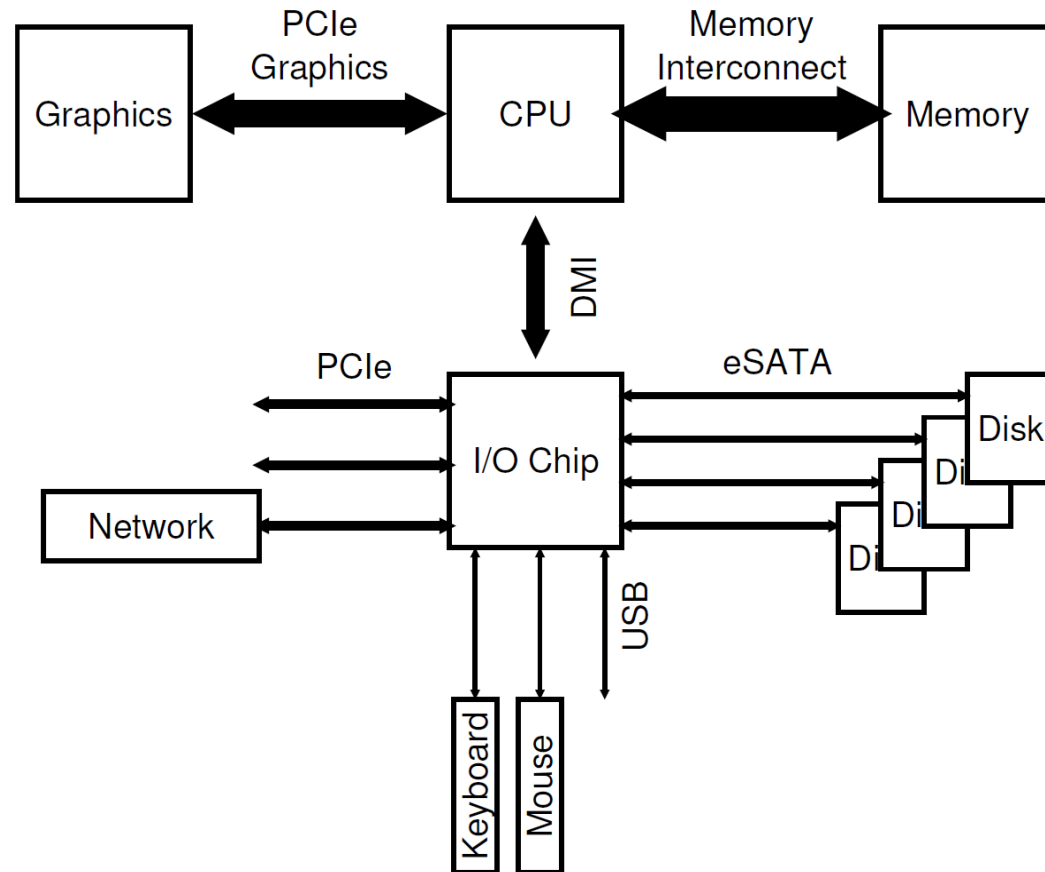
- But... thousands of devices, each slightly different
 - » OS: How can we **standardize** the interfaces to these devices?
- Devices unreliable: media failures and transmission errors
 - » OS: How can we make them **reliable**???
- Devices unpredictable and/or slow
 - » OS: How can we **manage** them if we don't know what they will do or how they will perform?

Simplified IO architecture

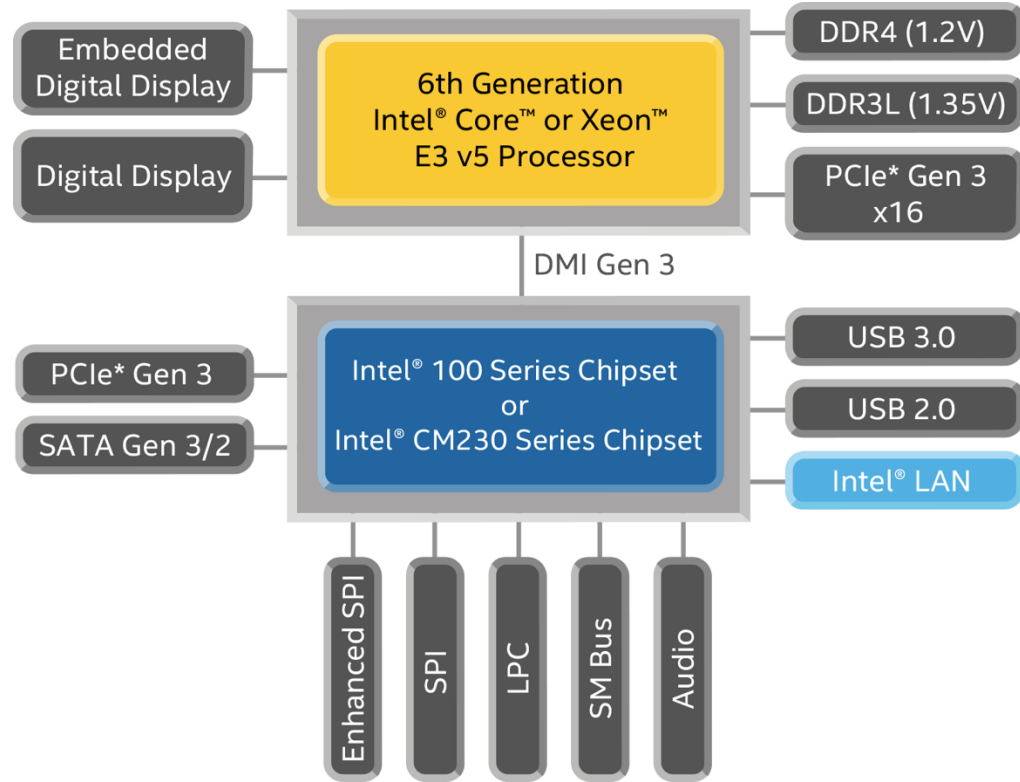


Follows a hierarchical structure because of cost: the faster the bus, the more expensive it is

Intel's Z270 Chipset



Sky Lake I/O: PCH



Sky Lake System Configuration

- **Platform Controller Hub**
 - Connected to processor with proprietary bus
 - » Direct Media Interface
- Types of I/O on PCH:
 - USB, Ethernet
 - Thunderbolt 3
 - Audio, BIOS support
 - More PCI Express (lower speed than on Processor)
 - SATA (for Disks)

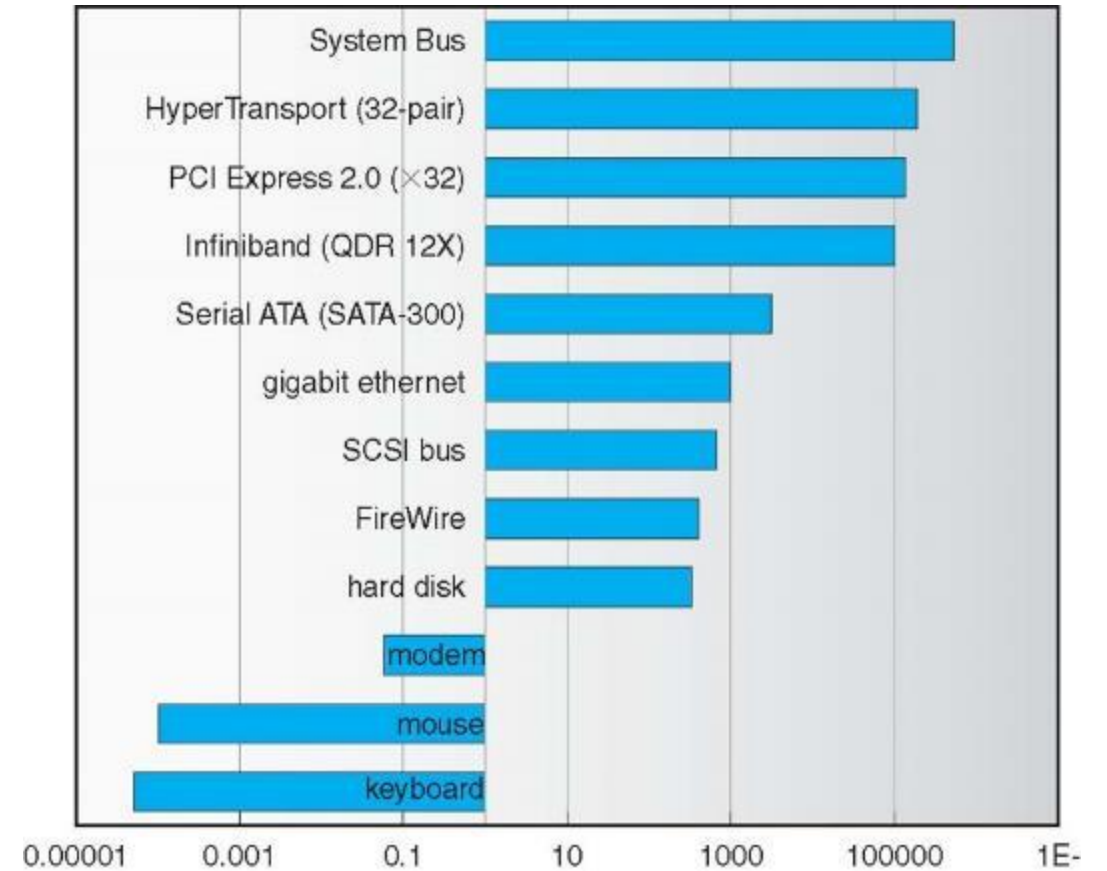
Wide Range of Timescales

**Jeff Dean:
“Numbers Everyone
Should Know”**

L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	25 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	3,000 ns
Send 2K bytes over 1 Gbps network	20,000 ns
Read 1 MB sequentially from memory	250,000 ns
Round trip within same datacenter	500,000 ns
Disk seek	10,000,000 ns
Read 1 MB sequentially from disk	20,000,000 ns
Send packet CA->Netherlands->CA	150,000,000 ns

Wide Range of Transfer Rates (Mb/s, Sun Enterprise 6000)

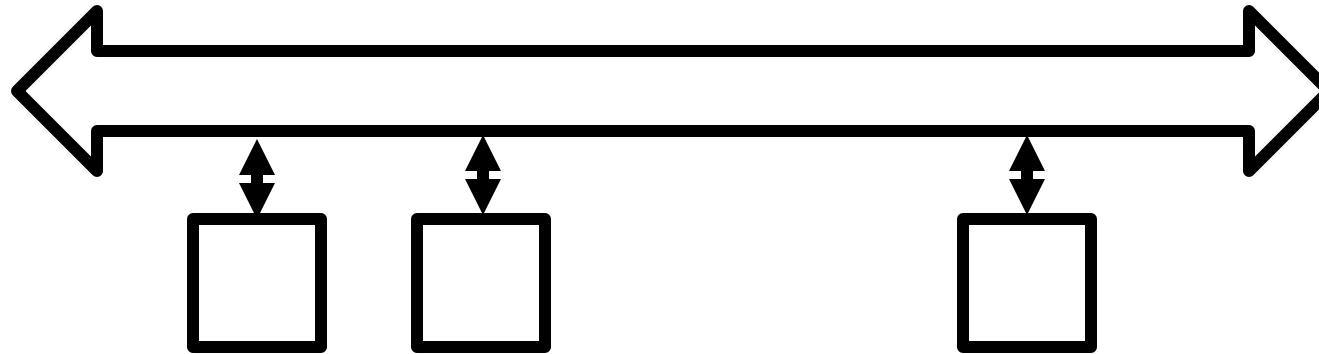
Device rates vary over 12 orders of magnitude!!!



Two questions

- What is a bus?
- How does the processor talk to the devices?

What's a bus?

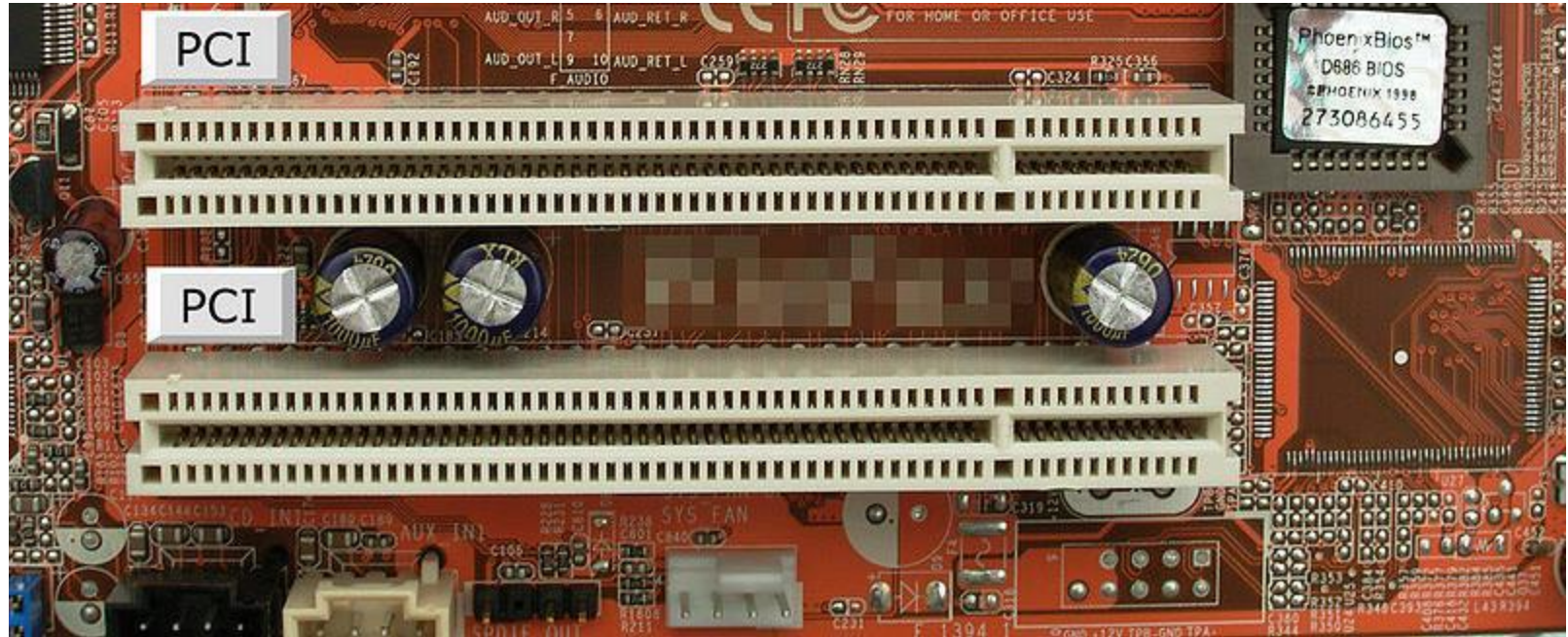


- Common set of wires for communication among multiple hardware devices plus protocols for carrying out data transfer transactions
- Split into three parts: control lines, address lines, and data lines
- Protocol: initiator requests access, arbitration to grant, identification of recipient, handshake to convey address, length, data
- High bandwidth close to processor, and slower but more flexible farther out

Why a Bus?

- Buses let us connect n devices over a single set of wires, connections, and protocols
 - $O(n^2)$ relationships with 1 set of wires (!)
- Downside: Only one transaction at a time
 - The rest must wait
 - “Arbitration” aspect of bus protocol ensures the rest wait

PCI Bus Evolution

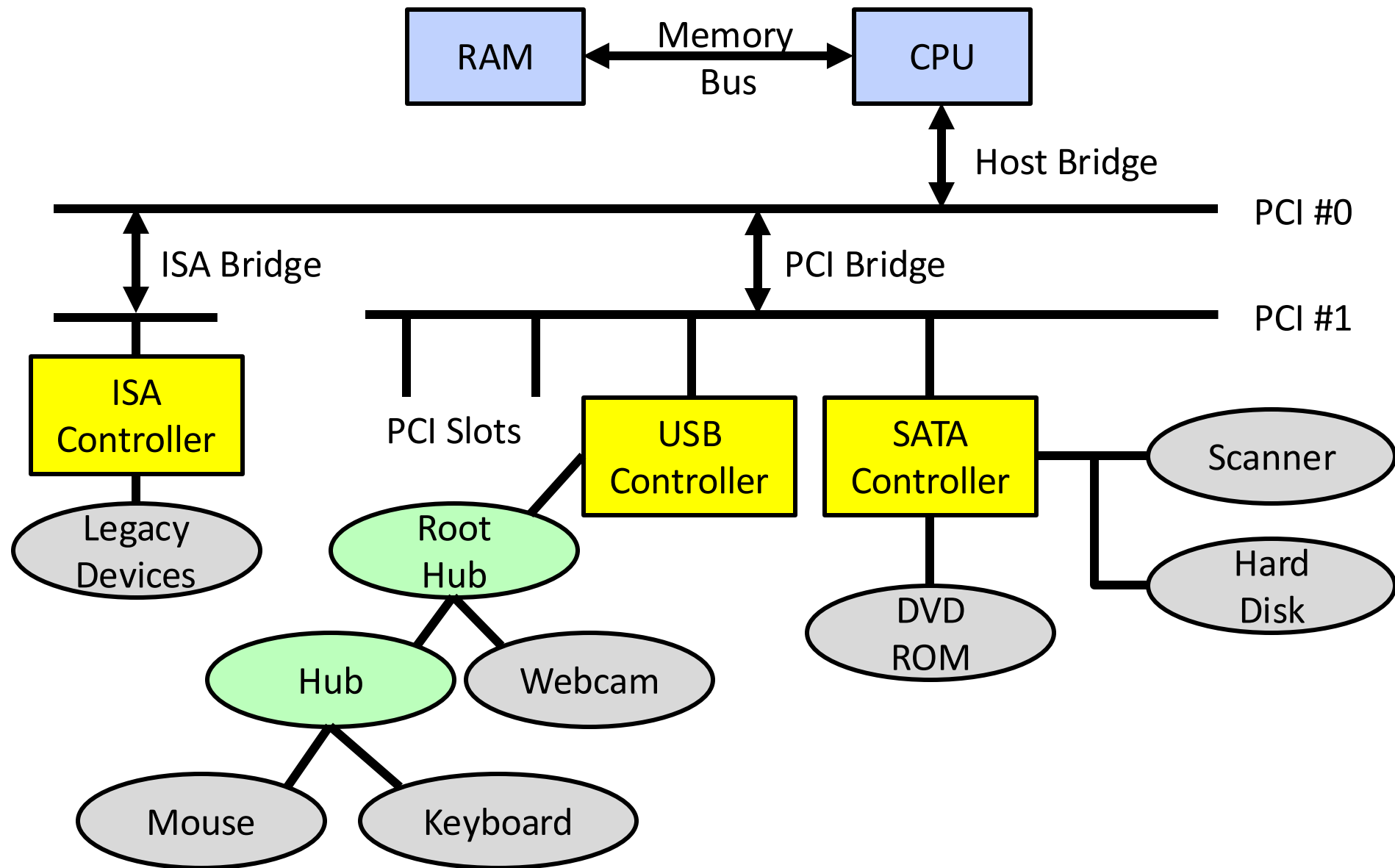


- PCI started life out as a parallel bus (send bits on many wires in parallel)
- But a parallel bus has many limitations
 - Hard to keep all the wires sending/receiving in sync
 - Slowest devices must be able to tell what's happening (e.g., for arbitration)
 - » Bus speed is set to that of the slowest device!

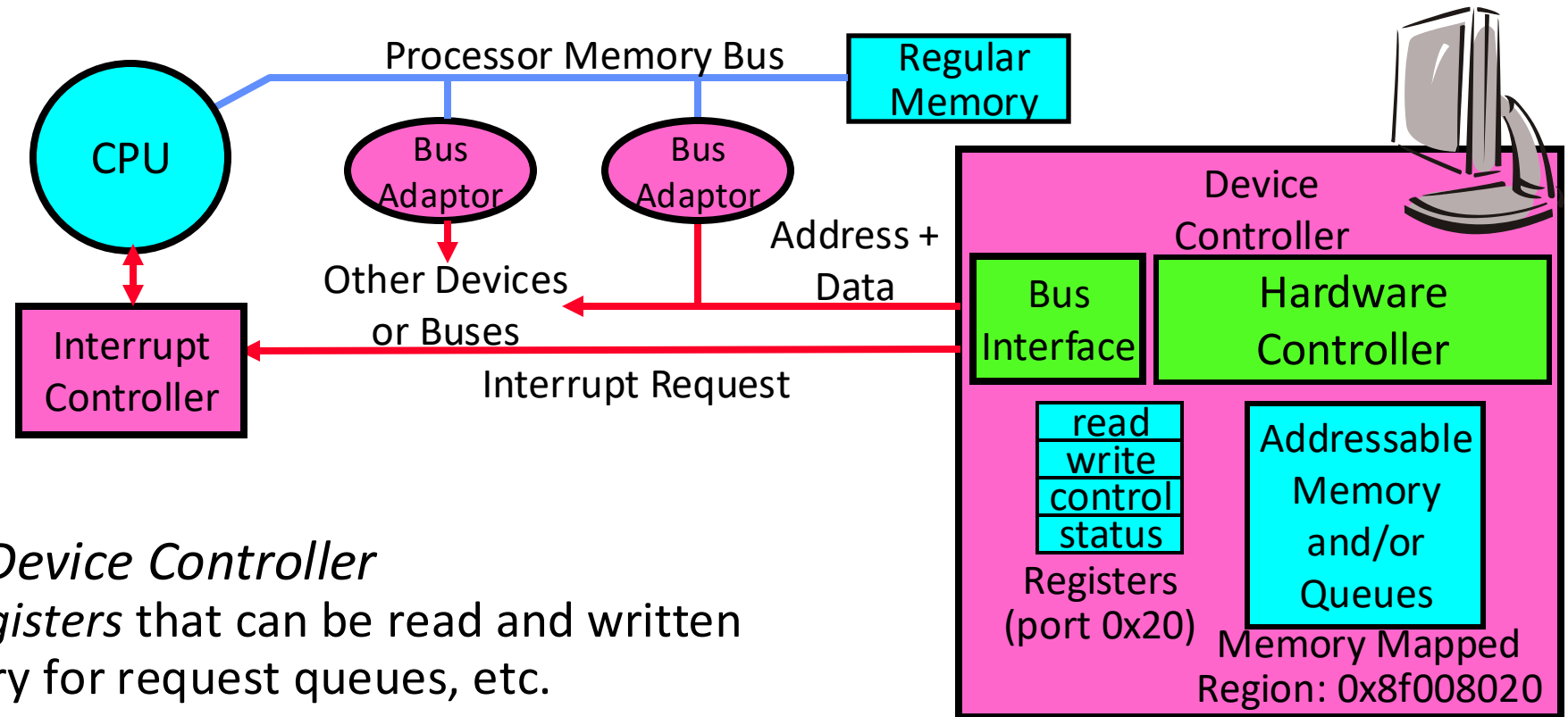
PCI Express (PCIe) “Bus”

- No longer a parallel bus
- Really a **collection of fast serial channels** or “lanes”
- Devices can use as many as they need to achieve a desired bandwidth
- Slow devices don’t have to share with fast ones
- One of the successes of device abstraction in Linux was the ability to migrate from PCI to PCI Express
 - The physical interconnect changed completely, but the old API still worked

Example PCI Architecture



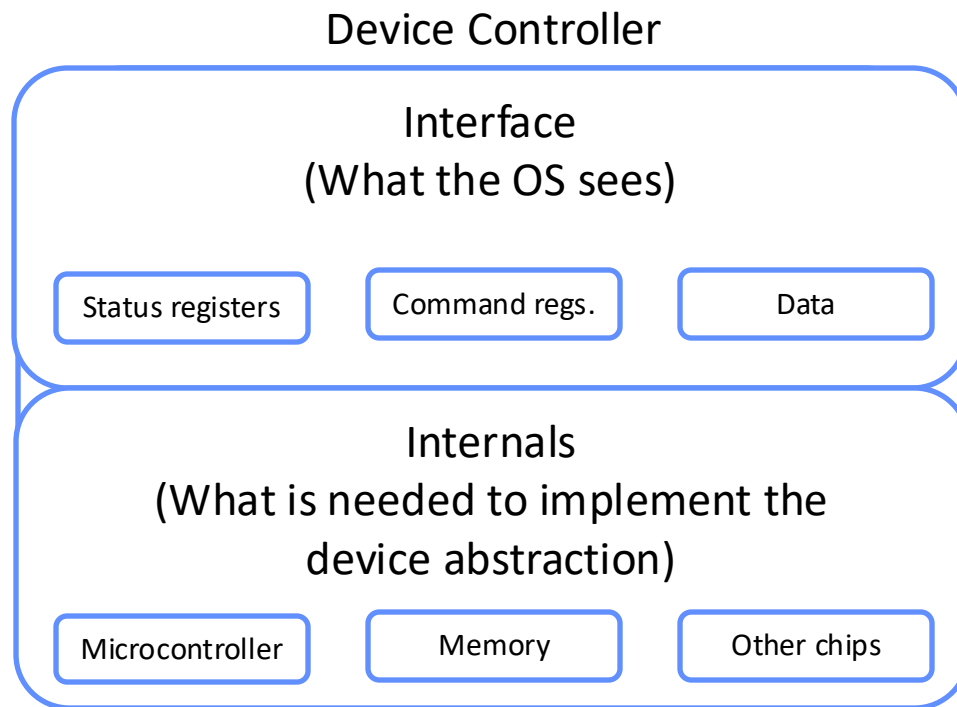
How does the Processor Talk to Devices?



- CPU interacts with a *Device Controller*
 - Contains a set of *registers* that can be read and written
 - May contain memory for request queues, etc.
- Processor accesses registers in two ways:
 - **Port-Mapped I/O**: in/out instructions
 - » Example from the Intel architecture: out 0x21,AL
 - **Memory-mapped I/O**: load/store instructions
 - » Registers/memory appear in physical address space
 - » I/O accomplished with standard load & store instructions

How does the processor talk to devices?

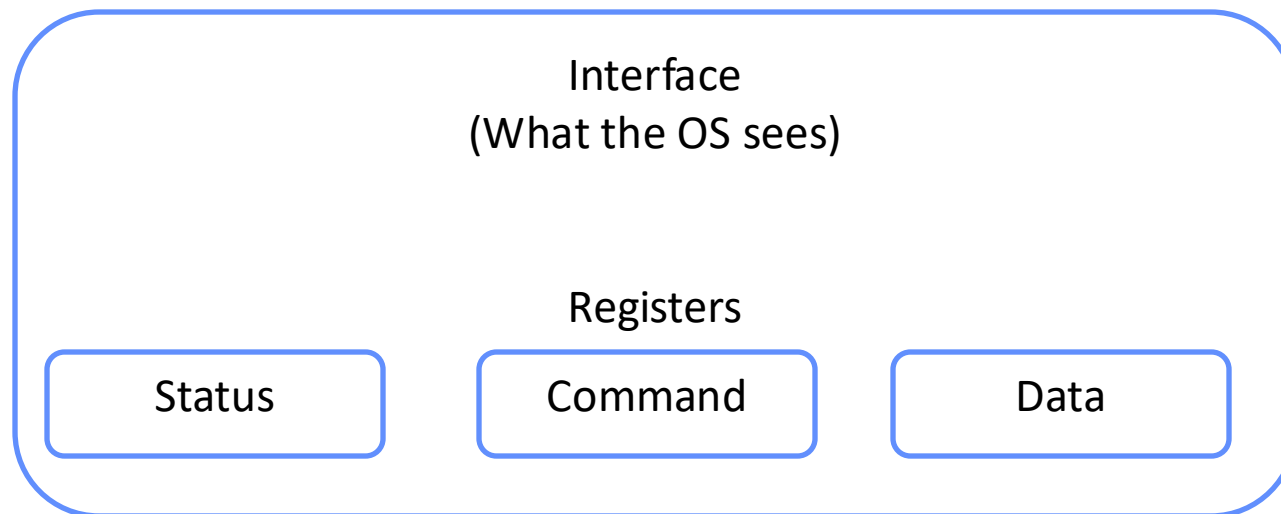
- Remember, it's all about abstractions!



Hardware interface device
presents to OS

Device implementation

How does the processor talk to devices?



Port-Mapped I/O:

Privileged in/out instructions

Example in Intel assembly: `out 0x21,AL`

Memory-mapped I/O: load/store instructions

Registers/memory appear in physical address space
I/O accomplished with load and store instructions

Example: Port-Mapped I/O in Pintos Speaker Driver

Pintos: devices/speaker.c

```
13  /* Sets the PC speaker to emit a tone at the given FREQUENCY, in
14     Hz. */
15  void
16  speaker_on (int frequency)
17  {
18      if (frequency >= 20 && frequency <= 20000)
19      {
20          /* Set the timer channel that's connected to the speaker to
21             output a square wave at the given FREQUENCY, then
22             connect the timer channel output to the speaker. */
23          enum intr_level old_level = intr_disable ();
24          pit_configure_channel (2, 3, frequency);
25          outb (SPEAKER_PORT_GATE, inb (SPEAKER_PORT_GATE) | SPEAKER_GATE_ENABLE);
26          intr_set_level (old_level);
27      }
28      else
29      {
30          /* FREQUENCY is outside the range of normal human hearing.
31             Just turn off the speaker. */
32          speaker_off ();
33      }
34  }
35
36  /* Turn off the PC speaker, by disconnecting the timer channel's
37     output from the speaker. */
38  void
39  speaker_off (void)
40  {
41      enum intr_level old_level = intr_disable ();
42      outb (SPEAKER_PORT_GATE, inb (SPEAKER_PORT_GATE) & ~SPEAKER_GATE_ENABLE);
43      intr_set_level (old_level);
44  }
```

Pintos: threads/io.h

```
64  /* Writes byte DATA to PORT. */
65  static inline void
66  outb (uint16_t port, uint8_t data)
67  {
68      /* See [IA32-v2b] "OUT". */
69      asm volatile ("outb %b0, %w1" : : "a" (data), "Nd" (port));
70  }
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

Example: Memory-Mapped Display Controller

- Memory-Mapped:

- Hardware maps control registers and display memory into physical address space

- » Addresses set by HW jumpers or at boot time

- Simply writing to display memory (also called the “frame buffer”) changes image on screen

- » Addr: 0x8000F000 — 0x8000FFFF

- Writing graphics description to cmd queue

- » Say enter a set of triangles describing some scene

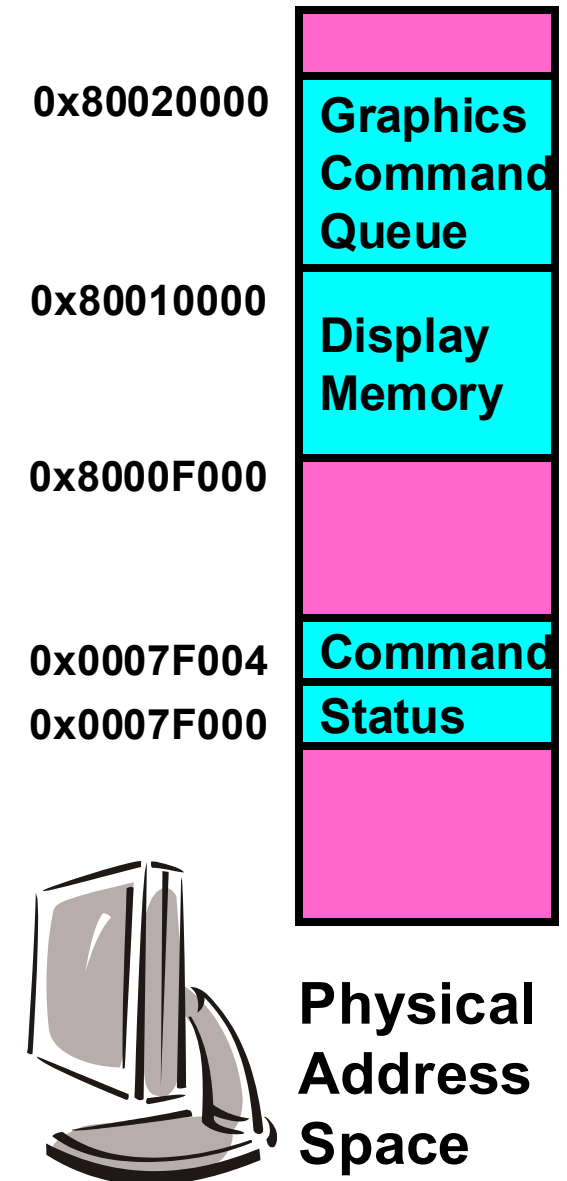
- » Addr: 0x80010000 — 0x8001FFFF

- Writing to the command register may cause on-board graphics hardware to do something

- » Say render the above scene

- » Addr: 0x0007F004

- Can protect with address translation



A Simple Protocol for Talking to a Device

```
While (STATUS == BUSY)
    ; // wait until device is not busy
Write data to DATA register
Write command to COMMAND register
    (starts the device and executes the command)
While (STATUS == BUSY)
    ; // wait until device is done with your request
```

Protocol does a lot of **polling**

CPU is responsible for moving data

How can we lower this overhead?

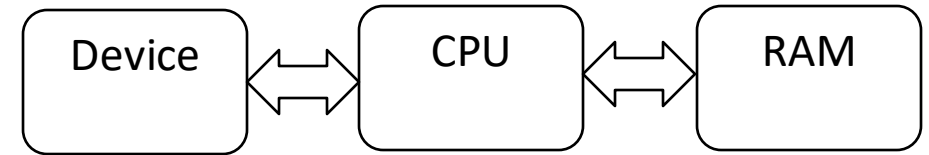
Interrupt-driven I/O vs Polling

- Use hardware interrupts to avoid busy polling:
 - Allows CPU to process another task. Will get notified when task is done
 - Interrupt handler will read data & error code
- Is it always better to use interrupts?
- Actual devices often support both polling and interrupts: e.g. wait for an interrupt from the network card to read the first packet, then poll its input queue memory space to look for other received packets

From programmed I/O to direct memory access

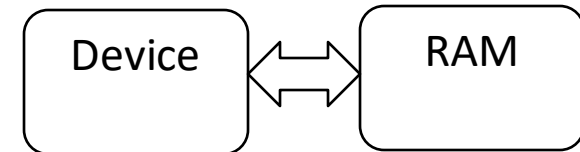
- With programmed I/O (our simple protocol):

- CPU issues read request
- Device interrupts CPU with data
- CPU writes data to memory
- Pros: simple hardware. Cons: Poor CPU is always busy!

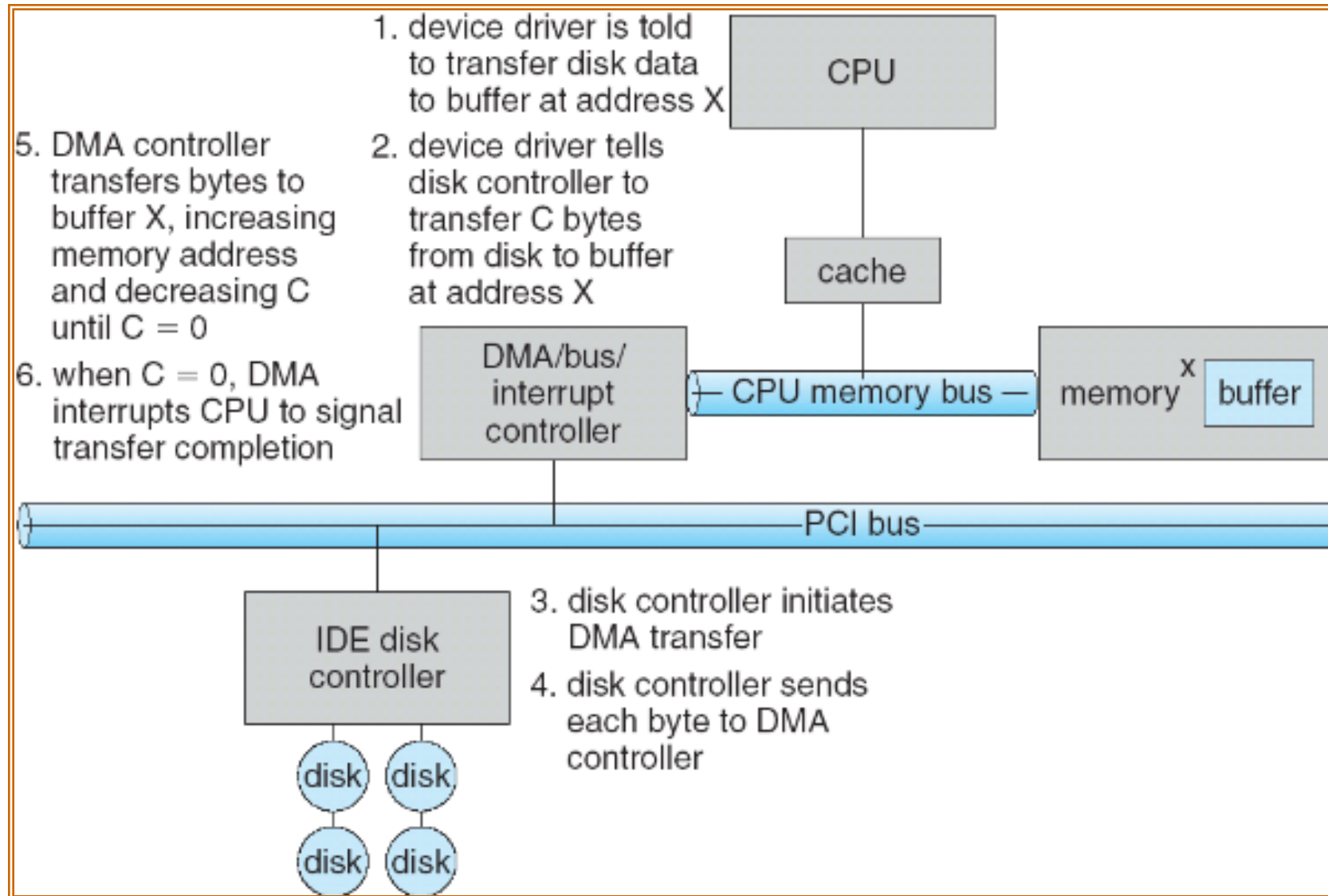


- With direct-memory-access (DMA):

- CPU sets up DMA request
 - » Gives controller access to memory bus
- Device puts data on bus & RAM accepts it
- Device interrupts CPU when done



DMA in more detail



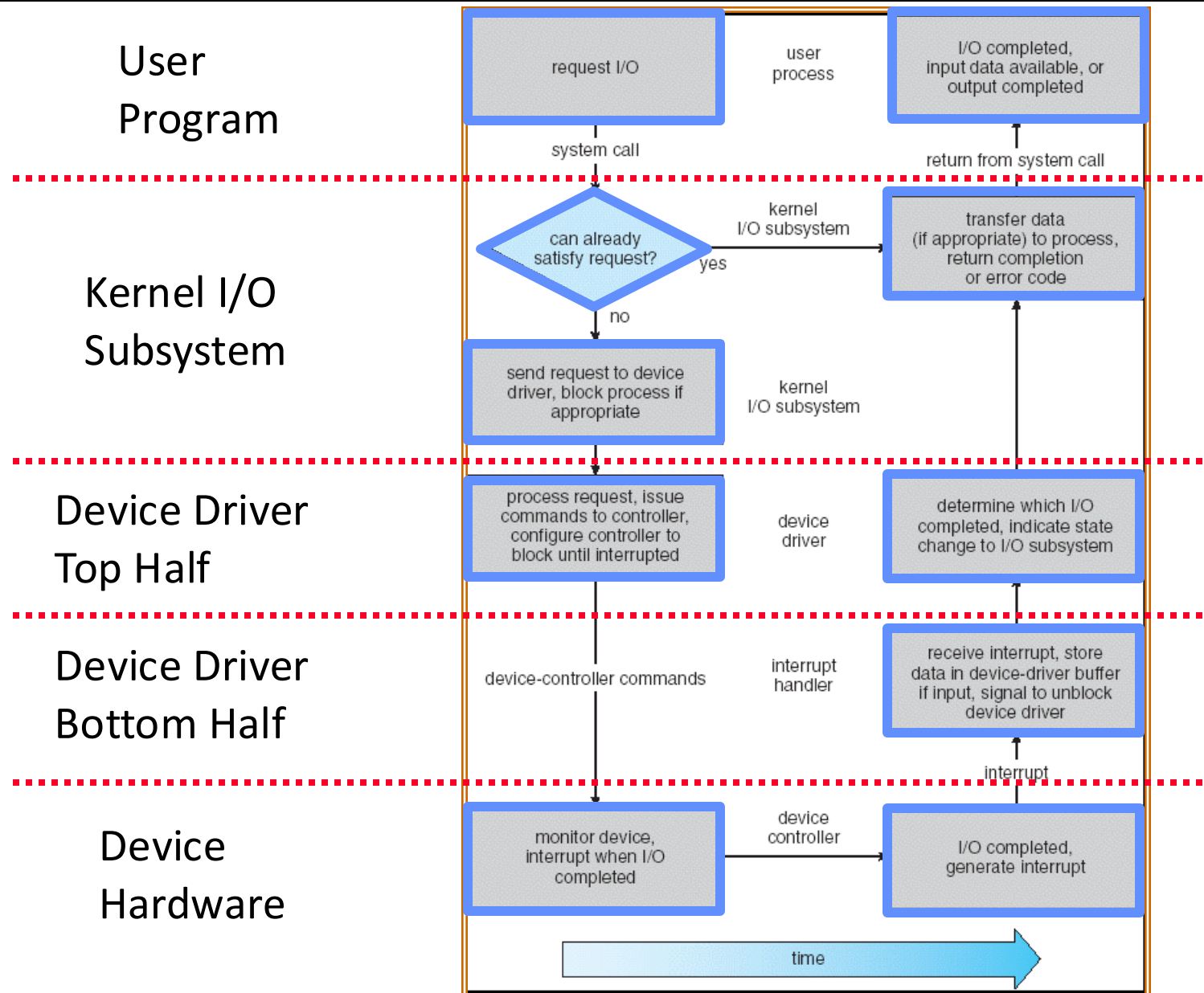
How can the OS handle ~~one~~ all devices

- How do we fit devices with specific interfaces into the OS, which should remain general?
 - Build a “device-neutral” OS and hide details of devices from most of OS
- Abstraction to the rescue!
 - **Device Drivers** encapsulate all specifics of device interaction
 - Implement device-neutral interfaces

Device Drivers

- **Device Driver:** Device-specific code in the kernel that interacts directly with that device hardware
 - Supports a standard, internal interface
 - Special device-specific configuration supported with the `ioctl()` system call
- Device Drivers are typically divided into two pieces:
 - **Top half:** accessed in call path from system calls
 - » implements a set of **standard, cross-device calls** like `open()`, `close()`, `read()`, `write()`, `ioctl()`, `strategy()`
 - » This is the kernel's interface to the device driver
 - » Top half will *start* I/O to device, may put thread to sleep until finished
 - **Bottom half:** run as interrupt routine
 - » Gets input or transfers next block of output
 - » May wake sleeping threads if I/O now complete
- Your body is 90% water, your OS is 70% device-drivers

Putting it together: Life Cycle of An I/O Request



Conclusion

- I/O Devices Types:
 - Many different speeds (0.1 bytes/sec to 50+ GBytes/sec)
 - Different Access Patterns:
 - » Block Devices, Character Devices, Network Devices
- Device Controllers: Hardware that controls actual device
 - Processor Accesses through I/O instructions (port-mapped I/O) or load/store to special physical memory addresses (memory-mapped I/O)
- Notification mechanisms
 - Interrupts
 - Polling: Report results through status register that processor looks at periodically
- Device drivers interface to I/O devices
 - Provide clean Read/Write interface to OS above
 - Manipulate devices through PIO, DMA & interrupt handling

How Does User Deal with Timing?

- **Blocking Interface:** “Wait”
 - When request data (e.g. `read()` system call), put process to sleep until data is ready
 - When write data (e.g. `write()` system call), put process to sleep until device is ready for data
- **Non-blocking Interface:** “Don’t Wait”
 - Returns quickly from read or write request with count of bytes successfully transferred
 - Read may return nothing, write may write nothing
- **Asynchronous Interface:** “Tell Me Later”
 - When request data, take pointer to user’s buffer, return immediately; later kernel fills buffer and notifies user (or lets user check/poll)
 - When send data, take pointer to user’s buffer, return immediately; later kernel takes data and notifies user (or lets user check/poll)

Storage Devices

Magnetic disks

- Storage that rarely becomes corrupted
 - Large capacity at low cost
- Block level random access (except for SMR – later!)
 - Slow performance for random access
- Better performance for sequential access

Flash memory

- Storage that rarely becomes corrupted
- Capacity at intermediate cost (5-20x disk)
 - Block level random access
- Good performance for reads; worse for random writes
 - Wear patterns issue